

כריית טקסט:

שיעור 1- מבוא לכריית טקסט וצורות ייצוג 04.03.2022:

- בכריית טקסט נתעסק בניתוח של טקסטים. כיצד מנתחים נתונים שאינם מובנים. אנו נתקלים בטקסטים בפוסטים, אימיילים, טוקבקים וכו'.
- ניתוח טקסט יכול להוות יתרון לארגון. באמצעות ניתוח טקסט, נוכל להפיק תובנות. למשל:
- בשירות לקוחות כשהלקוח כותב פניה, הפניה מגיעה למחלקה הנכונה רק לאחר זמן מה. אנו מצפים שיהיה ניתוב חכם ואוטומטי.
 - עובדי HR לא פותחים קו"ח לפני סינון של המחשב. המחשב מחפש מילות מפתח מתאימות.
 - בתחום השיווק אנו אוספים נתונים הקשורים למוצר שלנו כדי לבדוק ביקורות. נוכל לדעת אם קמפיין שיווקי הצליח.
 - בתחום הבריאות נוכל לדעת האם חולה יאושפז שוב בטווח של 30 ימים.
 - בתחום משפטים נוכל למצוא את המסמכים הרלוונטיים.

ההבדל בין כריית דאטה לכריית טקסט:

- בכריית דאטה התעסקנו בנתונים שהם מובנים. בכריית טקסט, ננתח טקסטים. איך מייצגים את הטקסט בצורה שהמחשב יבין אותו. נשתמש בנתונים לא מובנים.
- NPL- Natural language Processing - עיבוד שפה טבעית. מנסים לדמות התנהגות של בנאדם. למשל ע"י צ'אט בוט, גוגל טרנסלייט. אלו משימות שלא נעבוד עליהן בכריית טקסט.
- Supervised / Unsupervised Learning:
- Supervised - בעיות שאנו יודעים את הפתרון שלהם. יש סט נתונים שיש תיוג שלו. למשל באיריסים יש לנו תכונות של הפרח ובסוף יש לנו עמודה שאומרת מה הסוג של הפרח. יש נתונים מתויגים. גם בעץ החלטות שעסק באלגוריתמי חיזוי שיש רגרסיה, יש עמודה עם התיוג. למשל האם הלקוח ייקח הלוואה או שלא.
- Unsupervised - כשאין את העמודה של התיוג. יש אוסף של נתונים ואנו רוצים לחלק אותן לקבוצות. למשל לקחת קובץ של מסמכים ולחלק אותן לקבוצות. K mean וקלסטרים.

Workflow:

1. הגדרת הבעיה ומטרות ספציפיות.
2. זיהוי הטקסט שאנחנו רוצים לעסוק ואיזה טקסט זמין לנו.
3. ניקיון הנתונים.
4. בניית הפיצ'רים.
5. ניתוח, הרצת האלגוריתמים והערכה עד כמה הם טובים.
6. הגעה לתובנות ומסקנות.

Text Representation - סוגים עיקריים של ייצוג טקסט:

- Bag of words - שיטה פשוטה. נותנת תוצאות קצת פחות טובות, אך היא יותר מהירה. כשרוצים לבצע פעולה מהירה ויעילה, נשתמש ב-bag of words.
- o Term frequency - בודקים כמה פעמים מילה מסוימת מופיעה במסמך. אם היא מופיעה הרבה, יש לה משקל גדול.
- o TF-IDF - נרמול לכמות הפעמים שהמילה מופיעה בטקסט. נותן חשיבות למילים שמופיעים הרבה במסמך מסוים, אבל יחסית נדירות בכל המסמכים.
- Word embeddings - שיטה מסובכת יותר וגם קשה להבנה. מונח המשמש לייצוג של מילים לניתוח טקסט, בדרך כלל בצורה של וקטור בעל ערך אמיתי המקודד את משמעות המילה כך שהמילים הקרובות יותר במרחב הווקטור צפויות להיות דומות במשמעותן.
- o Word2vec - ייצוג מילה מסוימת כווקטור. ברגע שהמילה מיוצגת כווקטור, אפשר להכניס לווקטור כל מיני מאפיינים נוספים של המילה. מילים דומות מיוצגות ע"י ווקטורים דומים. יש שיטה לקחת אוסף של ווקטורים ולחשב ממוצע של הווקטורים ולייצג את המסמך כווקטור.

Bag of words

התייחסות למשפט כסל עם אוסף של מילים. אין חשיבות לסדר המילים. למשל, שני מסמכים. מילה מיוצגת בטבלה בעמודה. למשל, המילה ג'ון מופיע פעם אחת במסמך 1 ופעם אחת במסמך 2. אנו רוצים לייצג את המשפטים בצורה של מטריצה. כל מסמך, מיוצג ע"י ווקטור. כל עמודה היא פיצ'ר. כל מילה, מיוצגת ע"י עמודה. בשלב זה, משתמשים ב-term frequency. סופרים כמה פעמים כל מילה מופיעה. ברגע שיש לנו ווקטור, נוכל לעשות פעולות חשבוניות כמו חיסור כדי לקבל איזשהו מדד לקרבה של המסמכים.

Bag of Words											
Document 1: John likes to watch movies. Mary likes movies too.											
Document 2: John also likes to watch football games.											
Doc	John	likes	to	watch	movies	Mary	too	also	football	games	
1	1	2	1	1	2	1	1	0	0	0	
2	1	1	1	1	0	0	0	1	1	1	

The number represents the term frequency

Bag of N-grams

הרחבה טבעית של שק מילים. משמש כדי לשמור על קשר מסוים. n-gram הוא רצף של n מילים.

Example:

Unigram - [let] [me] [explain] [with] [an] [example]

Bigram - [let me] [me explain] [explain with] [with an] [an example]

Trigram - [let me explain] [me explain with] [explain with an] [with an example]

ב- R אנחנו מייצרים אובייקט שנקרא (Document Term Matrix (DTM מטריצה שמכילה את כל שורה מכילה מסמך והעמודות אלו המילים.

למשימות מסוימות נהפוך את המטריצה ל- (Term Document Matrix (TDM. כלומר, כל רשומה זו מילה וכל עמודה זה מסמך. יש אלגוריתמים שמצפים למסמך לאורך ויש כאלה לרוחב.

TDM sparsity and Term sparsity - מטריצה שיש לה הרבה אפסים. בגלל שאנחנו מייצגים כל מילה בכל הטקסטים, נוצרות מטריצות שהן sparse.

TDM sparsity and Term sparsity

	1	2	3	4	5
R	0	0	1	1	0
Text	0	0	1	0	0
Stats	1	1	0	0	0
Mining	0	1	1	0	0
Book	0	0	0	0	1

$$\text{Sparsity}(TDM) = \frac{\text{Number of zeros}}{\text{Number of cells}}$$

$$\text{Sparsity}(Term) = \frac{\text{Number of zeros for the term}}{\text{Number of documents}}$$

$$\text{TDM sparsity} = 17/25 = 0.68$$

$$\text{The term "Mining" sparsity} = 3/5 = 0.6$$

$$\text{The term "Book" sparsity} = 4/5 = 0.8$$

שיעור 2- מבוא לכריית טקסט וצורות ייצוג 11.03.2022:

הגדרות בסיסיות:

נשתמש בפקודת setwd כדי שהתוכנה תדע מאיפה נייבא מסמכים.
 ("שולחן העבודה\בר אילן\כריית טקסט\שיעור 2\OneDrive\idopo\C:\Users\setwd("C:\\Users\\idopo\\OneDrive\\2

String as factor = false. פקטור זה משתנה קטגוריאלי. אנו רוצים לנתח טקסטים ולכן נרצה שהטקסט יישאר כטקסט ולא יהפוך למספרים. נרשום את פעולה זו כשורה בפני עצמה כדי שהיא תהיה תקפה לכל קטעי הקוד באותו הקובץ.
 options(stringsAsFactors=FALSE)

במערכת יש כל מיני הגדרות של שפה, נשמור על אחידות בשפה האנגלית. במידה והיינו רוצים לדעת באיזה שפה המחשב נשתמש ב get locale.

```
Sys.setlocale("LC_ALL", "C")
```

```
Sys.getlocale()
```

ייבוא ספריות

```
library(stringi)
```

```
library(stringr)
```

קריאת קובץ, הדפסת 6 השורות הראשונות, צפייה ב- 4 העמודות הראשונות כווקטור, מספור כמות השורות, סיכום הקובץ, בדיקת סוג הטקסט (טקסט) והדפסת 6 השורות הראשונות של עמודת טקסט.

```
# read csv
```

```
text.df<-read.csv("oct_delta.csv")
```

```
head(text.df)
```

```
head(text.df[,c(1:4)])
```

```
nrow(text.df)
```

```
summary(text.df)
```

```
class(text.df$text)
```

```
head(text.df$text)
```

פונקציות טקסט בסיסיות:

1. nchar - ספירת מס' התווים

```
# nchar - number of characters
```

```
name <- "Ido"
```

```
nchar(name)
```

2. ספירת כמה תווים יש בכל אחת מהרשומות

3. גישה למקום ספציפי ב DF (שורה 4, עמודה 5)

```
nchar(text.df$text[1:10])
```

4. אורך מיקום ספציפי

```
text.df[4,5]
```

5. יצירת DF שיספור את מס' התווים בכל עמודת טקסט

```
nchar(text.df$text)
```

6. אורך ממוצע של מס' התווים בעמודת טקסט

```
nc<-data.frame(text = text.df$text,nchar = nchar(text.df$text))
```

7. subset - ניקיון בקובץ. הוצאת גורמים מיותרים. במקרה הזה, השארת תווים גדולים מ-0

```
mean(nchar(text.df$text))
```

8. paste - פונקציה שמשרשרת מחרוזות עם הפרדה באמצעות התו " | "

```
subset.doc <- subset(text.df, nchar(text.df$text)>0)
```

9. Paste0 - ללא הפרדה בין התווים

```
paste(c('a', 'b', 'cd'), collapse='|')
```

10. Substr - שליפה של חלק מהמחרוזת. בדוגמא, שליפת התווים 1-4

```
paste0('a', 'b', 'cd')
```

substr - extract a sub-string in a character vector

```
str <- "abcdefg"
substr(str, 1, 4)
```

יצירת משתנה שיבדוק מה שתי התווים האחרונים

```
num <- 2
last <- substr(str, nchar(str)-num+1, nchar(str)) # two last characters
```

11. דריסת חלק מהמחרוזת והחלפתה במשהו אחר. למשל, החל מהתו ה-5, נשים ...

התוצאה -abcd אם ננסה להוסיף עוד נקודה נקבל את אותה התוצאה כי אין 4 תווים אחרי הספרה d.

substring - replace a sub-string in a character vector

```
substring(str, 5) <- c("...")
str
```

12. grep - מציאה. למצוא בטקסט תו כלשהו. בדוג', יש 3 משפטים ואנו רוצים למצוא את המילה אנגליה, התשובה תהיה 1,3. אם נשים "F", R" יתעלם מאותיות קטנות, התשובה תהיה 1.

grep - find

```
sentences <- c("England is Beautiful", "just an example", "england left the EU")
grep("England", sentences, ignore.case = T)
```

13. gsub - מציאה והחלפה. מוצא מילה מסוימת ומחליף אותה באחת אחרת. במקום אנגליה □ UK. בדוג' רק אנגליה עם אות גדולה התחלף ל- UK כי כך הוגדר. אם נרצה שאנגליה תוחלף ל- UK גם כשהאות E קטנה וגם גדולה, צריך להוסיף בסוף את □ , (ignore.case = T)

gsub - find and replace

```
sentence <- "England is Beautiful. england left the EU"
gsub("England", "UK", sentence)
```

14. strsplit - הפרדת המחרוזת לפי תו מסוים. בדוג', הפרדת המחרוזת לפי התו אחוז.

strsplit - split strings

```
x <- "get%better%every%day"
xs <- strsplit(x,split = '%')
```

15. `unlist` – לאחר הרצה בקונסול, בקונסול יופיע בתור אובייקט של רשימה `[[1]]` כדי לעבוד עם המחרוזת, נרצה להוסיף אותה כעמודה או פעולה אחרת, נרצה להחזיר אותה לוקטור. לאחר הרצה בקונסול, בקונסול יופיע אובייקט `[1]`.

`# turns to vector`

```
unlist(xs)
```

16. יצירת פונקציה שתחזיר את `x` התווים האחרונים – שם הפונק' `last.chars`. הפונק' מקבלת 2 ארגומנטים- `text` (עמודת הטקסט) ו- `num` (מס' התווים שנרצה לשלוף). השורה השנייה היא מה שאנו רוצים לשלוף- `x` התווים האחרונים. בשורה האחרונה אנו אומרים לפונק' מה להחזיר.

`# agent workload`

```
last.chars<-function(text,num){  
  last<-substr(text, nchar(text)-num+1, nchar(text))  
  return(last)  
}
```

בחלק הראשון יצרנו את הפונקציה, בחלק השני נקרא לה. נרצה להפעיל את הפונק' על המחרוזת "R text..." ונרצה לקבל את 5 התווים האחרונים. בשורה השנייה נרצה להפעיל את המחרוזת על 2 השורות הראשונות ותפעיל את הפונק' `last.chars` על 2 התווים האחרונים. נפעיל את פונק' `table` הסופרת כמה תאים יש לי מערך מסוים. נקבל לכל נציג שירות כמה הודעות (רשומות) יש לנציג.

```
a<-last.chars('R text mining is great',5)  
last.chars(text.df$text[1:2],2)  
table(as.factor(last.chars(text.df$text,2)))
```

ייצוג הטקסט באמצעות corpus (אוסף מסמכים):

```
setwd("C:\\Users\\lidopo\\OneDrive\\2 טקסט\\שיעור  
library(tm)
```

`#Set Options`

```
options(stringsAsFactors=FALSE) #don't treat strings as factors (categories)  
Sys.setlocale('LC_ALL','C')
```

אנו רוצים לייצר את הטקסטים שלנו כ- `bag of words`. ראשית, ניצור ווקטורים משפטיים.

`### Document Term Matrix`

`# DTM`

```
text <- c("This is a small example, the word small appears twice", "This is the  
second one")
```

יצירת corpus:

1. כדי לייצר את האובייקט שספריית `tm` יודעת לעבוד איתו, נצטרך להפעיל את הפונקציה שנקראת `Vcorpus`- ווקטור של טקסטים.

`# create corpus - vector source`

```
corpus <- VCorpus(VectorSource(text))
```

2. יצירת קורפוס מ- `DF`. ראשית ניצור `DF` לדוג'. בשורה הבאה חייבים להוסיף עמודה של `"doc_id"` עם מספור של העמודות ע"י פונקציית `sequence` (סדרה) מ- 1 ועד המס' הסופי של הרשומות. לבסוף, ניצור את הקורפוס ע"י הפעלת הפונקציה `vcorpus` כחלק מה- `DF`.

`# another option - create corpus from data frame source`

`# first column MUST be named doc_id`

```
df <- data.frame(txt = text)  
df <- data.frame(doc_id=seq(1:nrow(df)),text=df$txt)  
corpus <- VCorpus(DataframeSource(df))
```

***לאחר יצירת הקורפוס נוכל להפעיל הרבה פונק' שהספרייה מאפשרת.

סוגי פונקציות של tm (text mining):

1. DTM - אובייקט מורכב שלא ניתן לראות בכולו בקונסול, ניתן לראות שיש 2 מסמכים, 9 מונחים, כמות ה- 0 ב- sparsity (ניתן לראות אותם ע"י הרצת השורה -inspect(DTM)). 7 תאים עם 0 ו- 11 ללא 0. לכן באחוזים יש 39% (7/18 sparsity).
2. TDM - האובייקט ההופכי שבו ניתן לראות את הנתונים לאורך ולא לרוחב.

Document Term Matrix

```
# DTM
DTM <- DocumentTermMatrix(corpus)
inspect(DTM)
as.matrix(DTM)
TDM <- TermDocumentMatrix(corpus)
as.matrix(TDM)
```

יש כמה דרכים למשקל ולנרמל. המטריצה הרגילה שלנו תשתמש ב- term TF (frequency).

weightTf

```
DTM<-DocumentTermMatrix(corpus, control=list(weighting=weightTf))
as.matrix(DTM)
```

במידה ונרצה לנרמל, נצטרך לעשות שינוי למשקל הבינארי שאומר שלא אכפת לנו כמה פעמים המילה מופיעה במסמך, אנו רק רוצים לדעת האם היא מופיעה- כלומר, יופיע 1 או 0.

weightBin

```
DTM<-DocumentTermMatrix(corpus, control=list(weighting=weightBin))
as.matrix(DTM)
```

לפעמים, נרצה להוריד את הגודל של המטריצה כי נוצרות מטריצות מאוד גדולות. ניצור ווקטור עם 4 משפטים, ניצור קורפוס, ניצור DTM, נסיף inspect כדי לראות שהוא נוצר בצורה תקינה.

remove sparse terms

```
text <- c("This is a small example", "This is another example", "example
appears many times", "small example")
corpus <- VCorpus(VectorSource(text))
DTM <- DocumentTermMatrix(corpus)
inspect(DTM) # 1 -> 12, 0 -> 16. sparsity = 16/(12+16)
```

כאן יש פונק' שנקראת removeSparseTerms - הורדת מונחים שמופיעים מעט פעמים. נשאיר מונח ש- sparsity שלו יהיה יותר קטן מ- 0.7.

Remove sparse terms from a document-term matrix.

```
smallDtm <- removeSparseTerms(DTM, .7) # 0.7 - the maximal allowed
sparsity per term.
inspect(smallDtm)
```

נרצה לייצר מסמכים שיש להם Bi-grams. נקרא לספריה RWeka.

##Bi-grams (tm)

```
library(RWeka)
library(tm)
```

נייצר פונקציה שנקראת BigramTokenizer. היא קוראת לפונק' ב- weka שנקראת NGramTokenizer. אנו מעבירים לו פרמטרים של מינימום ומקסימום. כלומר האורך הבסיסי יהיה 1 והמקסימום 2. כלומר שילוב של unigrams ו- bigrams. אם היינו רוצים trigram היינו רושמים 3.

בשורה השנייה נפעיל את ה-DTM, בתוך הפרמטר (control) שהוא מקבל, תעשה טוקניזציה (יצירת העמודות) ותפריד את המסמכים לפי מה שיש ב-bigrams

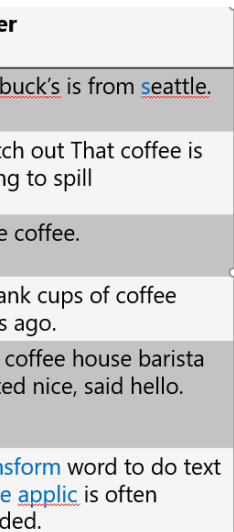
```
BigramTokenizer <- function(x) NGramTokenizer(x, Weka_control(min = 1,  
max = 2))  
DTM_BGram <- DocumentTermMatrix(corpus, control = list(tokenize =  
BigramTokenizer))  
inspect(DTM_BGram)
```

שיעור 3- הכנת נתונים וויזואליזציה 25.03.2022

שלב של עיבוד מראש עבור bag of words:

המשימות הנפוצות ביותר:

- הפיכת אותיות גדולות לקטנות
- הסרת סימני פיסוק, הסרת רווחים מיותרים
- הסרת מספרים
- הסרת "מילות עצר"
- הבאת המילה לצורה הבסיסית (יכול להיות שורש ויכול להיות קיצור של המילה)



נשתמש בספריית TM כדי לנתח טקסטים. לוקחים אוסף של מסמכים והופכים אותו לקורפוס.

כדי להפוך אותו לקורפוס, צריך להגיד מה המקור של המסמך. האם הקלט הוא ווקטור של מסמכים או הקלט הוא DF. לאחר ההחלטה נוכל להפעיל פונקציות באמצעות מיפוי (map).

- Text template

Document
source(s)

Corpus A

tm_map()

```
> # Make a vector source: my_source
> my_source <- VectorSource(my_tweets)

> # Make a volatile corpus: my_corpus
> my_corpus <- VCorpus(my_source)

> # Apply various preprocessing functions
> corpus <- tm_map(my_corpus, removeWords, custom.stopwords)
> corpus <- tm_map(my_corpus, removePunctuation)
> corpus <- tm_map(my_corpus, content_transformer(replace_abbreviation))
```


יש 3 צורות שונות להביא את המילים לצורה הבסיסית שלהם.
 Stem - חלק של מילה שקיצצו לה את ההתחלה / הסוף והביאו למילה מקוצרת שלא דווקא קיימת.
 Root - שורש של המילה. מילה מקוצרת.
 Lemma - צורה בסיסית של מילה, המילה חייבת להיות חוקית ולהופיע במילון. הוא פחות יעיל כי המילה חייבת להיות חוקית, האלגוריתם ניגש למילון כדי לבדוק את זה.

Stem vs. Root vs. Lemma

Stem	Root	Lemma
The part of the word that is common to all its inflected variants	Primary lexical unit of a word, cannot be reduced into smaller constituents	The canonical form, dictionary form, or citation form of a set of words
stem + affixes = valid word	doesn't have to be a word	valid word
Example	Example	Example
Words: produce, production	Word: interrupt	Words: fly, flies, flew, flying
Stem = <u>produc</u>	Root = <u>rupt</u>	Lemma = fly

ביצוע Stemming מתבצע באמצעות פונק' שנקראת `stemDocument`, לפונק' נותנים וקטור של מילים. למשל, הרצף הבא יהפוך ל- `complic`.
`stem_words->stemDocument(c("complicatedly", "complicated", "complication"))`

לאחר הפיכת המילים למילה מקוצרת, יכול להיות שנקבל מילה לא חוקית. נשתמש בפונק' `stemCompletion` על מנת להפוך את המילה לחוקית. למשל, `complic` יהפוך ל- `complicate`.
`stemCompletion(stem_words, c("complicate"))`

השלמה למילים לפי קורפוס. כלומר, לפי אוצר המילים של הקורפוס.
`stemCompletion(stem_words, my_corpus)`

ביצוע Lemmatization מתבצע ע"י ספריית `textstem` המכילה את הפונק' של `lemmatize_words`. אם ניתן את אותם מילים כמו בדוג' הקודמת `"complicatedly"` `"complication"` `"complicate"`.

אחד היתרונות הוא במקרים כמו `run` ו- `ran`. האלגוריתם יידע להפוך את צורת העבר לצורת ההווה.

```
library(textstem)
Vector -> c("complicatedly", "complicated", "complication")
lemmatized_words -> lemmatize_words(vector)
```

שימוש בקורפוס

```
corpus -> tm_map(corpus, lemmatize_strings)
```

שיטות נוספות לניקיון:
 • `'textclean' package'`

- `add_comma_space` - מוסיפה רווח במקום פסיק. טוב למקרה ששוכחים לשים רווח.
- `replace_emoticon` -0: מזהה הפונק' מזהה אמוטיקון ומחליפה אותו לתרגום מילולי.
- `replace_emoji` - מזהה אימוג'י ומחליפה אותו לתרגום מילולי.
- `replace_email` - מזהה מבנה של אימייל ומחליף אותו למבנה הרצוי.
- `replace_url` - מזהה מבנה של url ומחליף אותו למבנה הרצוי.

ביטוי רגולרי:

לרשום כלל שמחליף לנו רצף של תווים ברצף תווים אחר. משמש עבור "מצא" או "מצא והחלף". למשל, נרצה לזהות את כל המיילים, כל מה שמתחיל בהאשטאג לשבילונה קבועה.

Regular Expression Syntax:

Syntax	Description
\d	Digit, 0,1,2 ... 9
\D	Not Digit
\s	Space
\S	Not Space
\w	Word
\W	Not Word
\t	Tab
\n	New line
^	Beginning of the string
\$	End of the string
\	Escape special characters, e.g. \ is "\", \+ is "+"
	Alternation match. e.g. /(e d)n/ matches "en" and "dn"
.	Any character, except \n or line terminator
[ab]	a or b
[^ab]	Any character except a and b
[0-9]	All Digit
[A-Z]	All uppercase A to Z letters
[a-z]	All lowercase a to z letters
[A-z]	All Uppercase and lowercase a to z letters
i+	i at least one time
i*	i zero or more times
i?	i zero or 1 time
i{n}	i occurs n times in sequence
i{n1,n2}	i occurs n1 - n2 times in sequence
i{n1,n2}?	non greedy match, see above example
i{n,}	i occurs >= n times
[alnum:]	Alphanumeric characters: [:alpha:] and [:digit:]
[alpha:]	Alphabetic characters: [:lower:] and [:upper:]
[blank:]	Blank characters: e.g. space, tab
[cntrl:]	Control characters
[digit:]	Digits: 0 1 2 3 4 5 6 7 8 9
[graph:]	Graphical characters: [:alnum:] and [:punct:]
[lower:]	Lower-case letters in the current locale
[print:]	Printable characters: [:alnum:], [:punct:] and space
[punct:]	Punctuation character: ! " # \$ % & ' () * + , - . / : ; < = > ? @ [\] ^ _ ` { } ~
[space:]	Space characters: tab, newline, vertical tab, form feed, carriage return, space
[upper:]	Upper-case letters in the current locale
[xdigit:]	Hexadecimal digits: 0 1 2 3 4 5 6 7 8 9 A B C D E F a b c d e f

- `gsub` function - Replaces all matches of a string

- Example:

• `gsub("https\\S*", "", tweets$text)`

• `gsub("@\\S*", "", tweets$text)`

\\S = Not Space
* = Any number of characters

What will be the result?

`[^\\sA-Za-z\\'\\-]`

[^] – any character except
\\s – space
A-Za-z – all the letters
\' = '
\\- = -

Identify all characters except - all letters (a-z and A-Z), """,
, "- and space

Cheat sheet -

<https://paulvanderlaken.files.wordpress.com/2017/08/r-regular-expression-cheatsheet.pdf>

```
setwd("C:\\Users\\idopo\\OneDrive\\3 טקסט\\שיעור")
library(tm)
library(stringi)
library(textstem)
```

```
### Text cleaning
### gsub example
```

בשורה זו נקבל רק מה שלאחר הרווח - "after the space".

```
gsub("https\\S*", "", "httpsandmore after the space")
```

בשורה זו נקבל מה שלפני השטרודל ומה שלאחר הרווח - "email after the space".

```
gsub("@\\S*", "", "email@address.com after the space")
```

בשורה זו נקבל את כל המילים וסימני פיסוק שאינם / או @ - "email address com".

```
gsub("[^\\sA-Za-z\\-\\-]", "", "email@address.com after-the/space's")
#advanced (keep all letters, ', -)
```

```
##### Dataframe source example
```

```
text.df<-read.csv('oct_delta.csv')
head(text.df$text)
```

כעת, נפעיל ניקיונות על כל הטקסטים. כל מה שיבוא אחרי @ / * יימחק (שמות הנציגים).

```
# regular expression
# remove all the characters after the @
text.df$text <- gsub("@\\S*", "", text.df$text)
# remove all the characters after the *
text.df$text <- gsub("\\*\\S*", "", text.df$text)
head(text.df$text)
```

ביצירת קורפוס עם DF, צריכה להיות עמודה של doc_id המכילה מספור כל המשפטים שיש.

```
# If using DataframeSource in VCorpus the first column MUST be named
doc_id followed by a text column
tweets<-data.frame(doc_id=seq(1:nrow(text.df)),text=text.df$text)
head(tweets)
```

יצירת וקטור של כל רשימת ה- stopwords באנגלית, נרצה להוריד את המילים- lol, smh, delta.

```
custom.stopwords <- c(stopwords("english"),'lol','smh','delta')
```

הפעלת כל הפונק' של הניקיון.
הפיכת כל האותיות הגדולות לקטנות, מחיקת stop words, מחיקת סימני פיסוק, מחיקת מספרים ומחיקת רווחים.

```
# clean the corpus
corpus <- VCorpus(DataframeSource(tweets)) #Dataframe source
corpus <- tm_map(corpus, content_transformer(tolower))
corpus <- tm_map(corpus, removeWords, custom.stopwords)
corpus <- tm_map(corpus, removePunctuation)
corpus <- tm_map(corpus, removeNumbers)
corpus <- tm_map(corpus, stripWhitespace)
```

אחרי כל הרצה נריץ שורה זו כדי לבדוק תקינות (במקרה הזה של השורה הראשונה בלבד).

one element in a list

```
corpus[[1]][1]
```

כדי לבדוק תקינות של 5 השורות הראשונות, נריץ את הלולאה הבאה.

several elements

```
for(i in 1:5){  
  print(corpus[[i]][1])  
}
```

ניצור עותק לקורפוס לפני שנבצע שינויים נוספים. לאחר מכן נריץ stemming כדי לקבל קיצורי מילים (לא כל המילים הן מילים אמיתיות).

```
copy_corpus <- corpus
```

Stemming

```
corpus <- tm_map(corpus, stemDocument) ##Stem words in a text document  
using Porter's stemming algorithm.
```

```
for(i in 1:5){  
  print(corpus[[i]][1])  
}
```

כדי להפעיל את ה- lemmatizing נפעיל את ספריית textstem, פונק' tm_map על עותק הקורפוס, ייקח קצת זמן לריצה בגלל שהאלגוריתם נאלץ לחפש מילים במילון.

Lemmatizing

```
library(textstem)  
corpus_lm <- tm_map(copy_corpus, content_transformer(lemmatize_strings))
```

```
for(i in 1:5){  
  print(corpus_lm[[i]][1])  
}
```

דוגמאות:

#####vector source example

#####stemming example

```
t <- c("The first sentence contains the words: complicated,  
complicated and complication",  
      "This is the second sentence, this task is very complicated. the surgery  
can cause complications.",  
      "The third sentence is: run, running, ran")
```

```
myCorpus <- VCorpus(VectorSource(txt)) #Vector source
```

clean the text

```
myCorpus <- tm_map(myCorpus, content_transformer(tolower))  
myCorpus <- tm_map(myCorpus, removePunctuation)
```

stemming

```
myCorpus <- tm_map(myCorpus, stemDocument) ##Stem words in a text  
document using Porter's stemming algorithm.
```

```
for(i in 1:3){  
  print(myCorpus[[i]][1])  
}
```

#####lemmatizing example

```
myCorpus <- VCorpus(VectorSource(txt))
```

clean the text

```
myCorpus <- tm_map(myCorpus, content_transformer(tolower))
myCorpus <- tm_map(myCorpus, removePunctuation)
myCorpus[[1]][1]
```

lemmatize the text

```
library(textstem)
myCorpus <- tm_map(myCorpus, content_transformer(lemmatize_strings))
for(i in 1:3){
  print(myCorpus[[i]][1])
}
```

##create a function

```
clean.corpus<-function(corpus){
  corpus <- tm_map(corpus, content_transformer(tolower))
  corpus <- tm_map(corpus, removeWords, custom.stopwords)
  corpus <- tm_map(corpus, removePunctuation)
  corpus <- tm_map(corpus, removeNumbers)
  corpus <- tm_map(corpus, stripWhitespace)
  return(corpus)
}
```

```
custom.stopwords <- c(stopwords("english"),'lol','smh','delta','amp') #words to
remove
```

#create a corpus

```
corpus <- VCorpus(DataframeSource(tweets))
```

#apply the cleaning function

```
corpus<-clean.corpus(corpus)
```

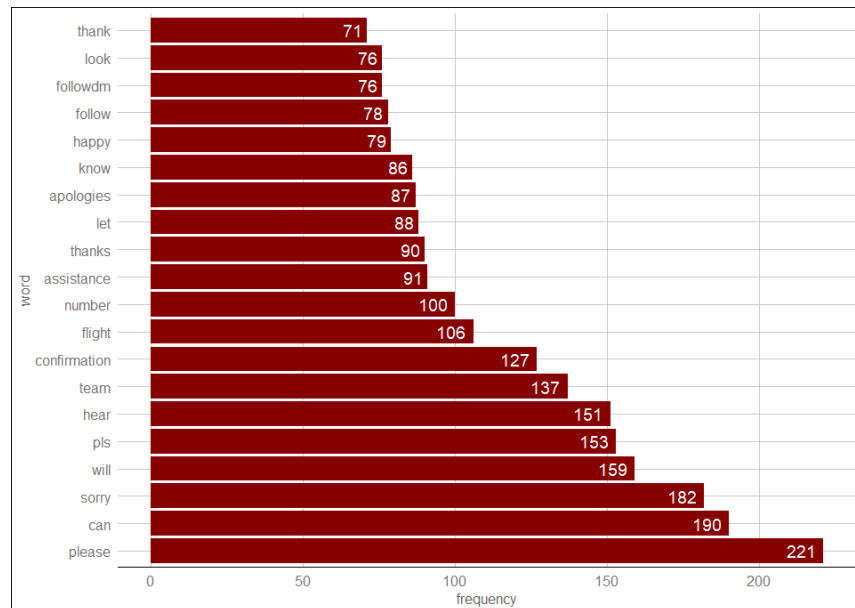
שיעור 4- הכנת נתונים וויזואליזציה 01.04.2022:

משתמשים בוויזואליזציה בשביל שני דברים. כאשר יש טקסט חדש ואנו רוצים לחקור אותו ובשביל ליצור ויזואליזציות כדי להציג את הנתונים שלנו, להציג למנהלים את המסקנות. סוגי ויזואליזציות:

1. Bar Plot of word frequencies - נראה כמה פעמים כל מילה מופיעה בטקסט. ניתן לראות שמדובר בברים, בציר X נראה את התדירות, ובציר Y את המילה. הצבע אדום כהה (darkred). משתמשים בפונקציה geom_bar כדי לצייר את הבר פלוט, נרצה לראות שכיחות של 20 המילים החוזרות ביותר. בנוסף, יש פונקציה שנקראת coord_flip בין הקורדינטות של Y ו X.

a bar plot of the first 20 terms

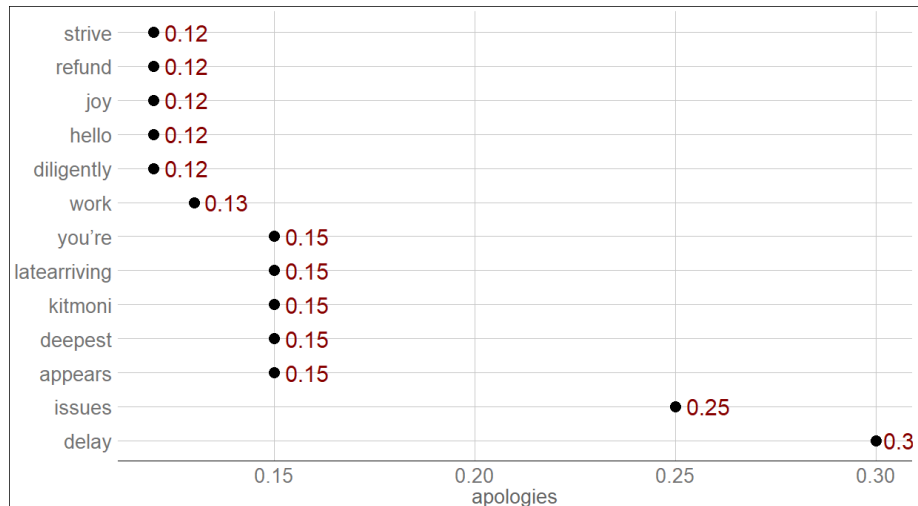
```
ggplot(freq.df[1:20,], aes(x=word, y=frequency)) +  
geom_bar(stat="identity", fill='darkred')  
+ coord_flip()
```



2. Word Association - אסוציאציה של מילים. ניתן לראות איזה מילים מופיעה באסוציאציה גבוהה עם המילה apologies. המילה delay מופיעה הכי הרבה פעמים בהקשר של apologies בין 0 ל- 1. נשתמש ב- scatter plot ע"י פונקציה שנקראת findAssocs. ניתן לה את tdm, את מילת המפתח וערך סף, כלומר לא נראה תוצאה פחותה מהערך סף.

#find the associations for the word 'apologies'. 11% is the association threshold

```
associations<-findAssocs(tdm, 'apologies', 0.11)
```



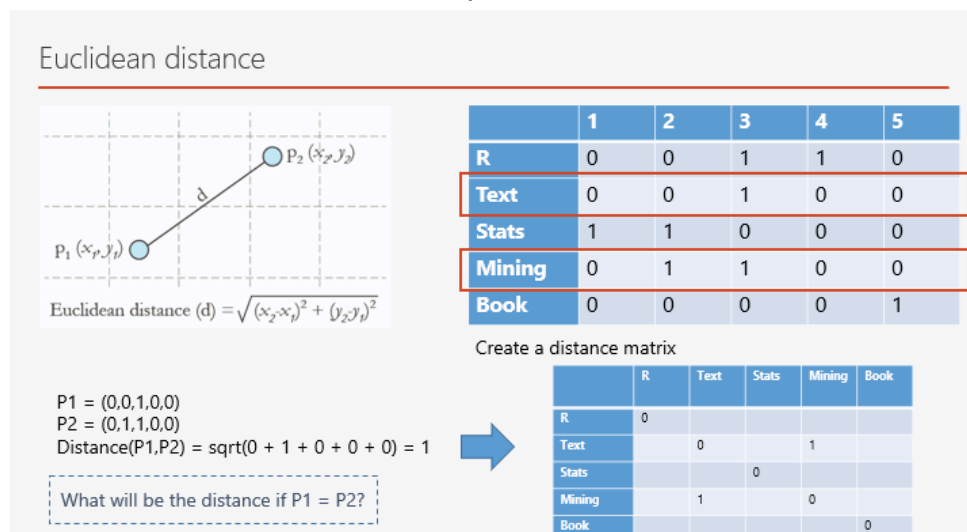
3. Dendrogram - מילים שמופיעות הרבה פעמים. צורת ייצוג של מילים שקרובות אחת לשנייה.

הדמיה דמוית עץ, מחשב את המרחק בין הווקטורים של המונחים. התנאים יהיו קרובים יותר אם הם יופיעו יחד במסמכים רבים.

כדי ליצור דינדרוגרמה נשתמש בפונקציה `hclust`, היא מקבלת מטריצת מרחקים בין מילה מסוימת לאחרת. חישוב מרחק בין וקטורים מתבצע באמצעות מרחק אוקלידי.

נשתמש ב- `tdm` על מנת להציג את המונחים (מילה בשורה) וכל מסמך הוא עמודה. כך נוכל לחפש את המרחק בין המילה `Text Mining` למילה `Book`.

כדי להציג את הדינדרוגרמה נשתמש בפונקציית `plot`.

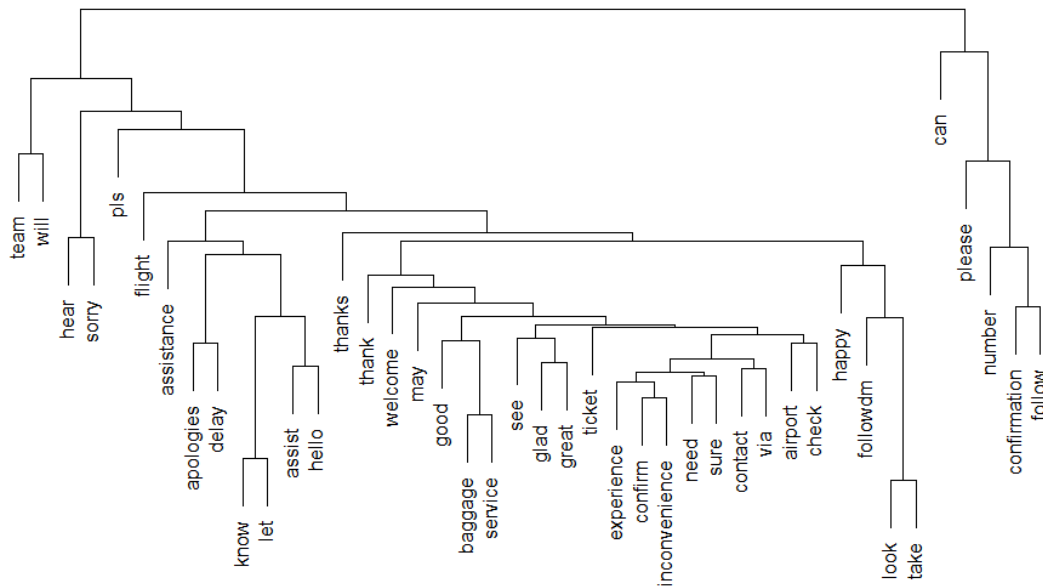


```
hc <- hclust(dist(tdm2, method = "euclidean"), method = "complete")
```

```
#plot the dendrogram
```

```
plot(hc, yaxt='n', main = '@DeltaAssist Dendrogram')
```


@DeltaAssist Dendrogram

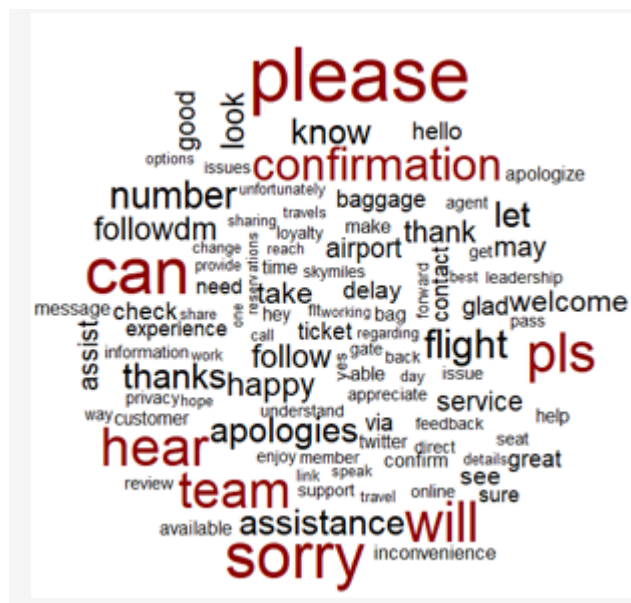


4. Word cloud - הדגמה של מילים בצורת ענן. הדמיה פופולרית מאוד, מבוסס על תדירות. גודל הגופן מתאים לתדירות המונחים. ככל שהמילה מופיעה יותר פעמים בטקסט, היא תהיה יותר גדולה. נשתמש בספריה ובפונקציה שנקראת wordcloud. אנחנו נותנים לפונק' חישוב מקדים לשכיחות לכל מילה. ***במידה ולא ננקה טוב את הטקסט, בענן יופיעו סימני ניקוד ודברים נוספים לא חשובים.

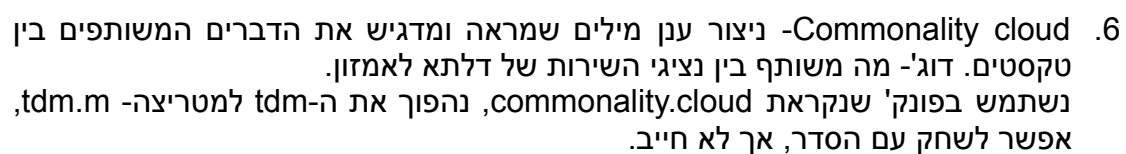
create a word cloud after we have the frequency table#

library(wordcloud)

wordcloud(freq.df\$word,freq.df\$frequency, max.words = 100,
('colors=c('black','darkred



```
#number of colors just like the number of different documents we compare
comparison.cloud(tdm.m, max.words = 200, random.order=FALSE, title.size
= 1, colors=brewer.pal(ncol(tdm.m), "Dark2"))
```



```
commonality.cloud(tdm.m, max.words = 200, random.order=FALSE,  
colors=pal)
```



7. Polarized cloud - הבדלים במרחק בין התדירות של מילים מסוימות בטקסט אחד לשני. ההבדלים הגדולים נמצאים למטה וככל שעולים, ההבדלים יותר קטנים. נשתמש בפונק' pyramid.plot, ניתן לה את כמות הפעמים שמילה מסוימת מופיעה אצל דלתא ואצל אמזון ואת כמות התוויות.

```
pyramid.plot(top25.df$x, top25.df$y, labels = top25.df$labels,
             gap = 30, top.labels = c("Amazon", "Words", "Delta"),
             main = "Words in Common")
```

#Text mining in R

#part 2.2 code - frequent terms and associations visualization

```
setwd("E:/Text mining course/2")
```

```
options(stringsAsFactors=FALSE)
```

```
Sys.setlocale("LC_ALL","C") #English(US)
```

נשתמש ב- ggplot2 כדי להציג את כל הגרפים בשביל ויזואליזציות. ggthemes לייצוג יפה יותר.

```
library(tm)
```

```
library(ggplot2)
```

```
library(ggthemes)
```

#read the dataset

```
text.df <- read.csv('oct_delta.csv')
```

```
tweets <- data.frame(doc_id=seq(1:nrow(text.df)),text=text.df$text)
```

prepare the data

#create a function to clean the corpus

```
clean.corpus <- function(corpus){
```

```
  corpus <- tm_map(corpus, content_transformer(tolower))
```

```
  corpus <- tm_map(corpus, removeWords, custom.stopwords)
```

```
  corpus <- tm_map(corpus, content_transformer(function(x) gsub(x, pattern =
"http\\S*", replacement = ""))) # remove all the characters after the http
```

```

corpus <- tm_map(corpus, content_transformer(function(x) gsub(x, pattern =
"@\\S*", "", replacement = ""))) # remove all the characters after the @
corpus <- tm_map(corpus, content_transformer(function(x) gsub(x, pattern =
"\\*\\S*", "", replacement = ""))) # remove all the characters after the *
corpus <- tm_map(corpus, removePunctuation)
corpus <- tm_map(corpus, removeNumbers)
corpus <- tm_map(corpus, stripWhitespace)
return(corpus)
}

```

```

custom.stopwords <- c(stopwords("english"), 'lol', 'smh', 'delta', 'amp') #words to
remove

```

```

#create a corpus

```

```

corpus <- VCorpus(DataframeSource(tweets))

```

```

#apply the cleaning function

```

```

corpus<-clean.corpus(corpus)

```

```

# view 10 first rows

```

```

for(i in 1:10){
  print(corpus[[i]][1])
}

```

נשתמש ב- qdap כשנרצה להפוך את הקורפוס בצורה קלה ל- DF. יעזור להסתכל על הניקיונות באקסל.

```

###until here - saw last time

```

```

# write the corpus to a csv file

```

```

library(qdap)
corpus.df <- as.data.frame(corpus)
write.csv(corpus.df, "corpus.df.csv")

```

כדי לייצר ויזואליזציה, נייצר קודם כל tdm.

```

#create a Term Document Matrix

```

```

tdm<-TermDocumentMatrix(corpus)

```

```

inspect(tdm[1:10, 1:5]) #inspect the result, 10 first terms, 5 documents

```

כעת, נהפוך את tdm למטריצה כדי להתאים את הצרכים למערכת

```

#convert to a matrix

```

```

tdm.tweets.m<-as.matrix(tdm)

```

נחשב את כמות הפעמים שהמילה מופיעה בטקסטים. נשתמש בפונק' rowSums שלוקחת את השורות וסוכמת את הערכים של השורות. לאחר הרצה נגלה כמה פעמים מופיעה כל מילה.

```

#summarize the rows (number of appearances of each term)

```

```

term.freq<-rowSums(tdm.tweets.m)

```

```

term.freq[1:10] # term frequency of the first 10 terms

```

התוצאות לא ממוינות לפי סדר, כדי לעשות זאת, נמין את השכיחות ל- DF עם שתי עמודות, עמודה ראשונה עם word והעמודה השנייה frequency עם הערך.

```

#create a dataframe with two columns, word and frequency

```

```

freq.df<-data.frame(word=names(term.freq),frequency=term.freq)

```

ניקח את ה- DF, נמין אותו לפי העמודה השנייה (שכיחות) ולפי סדר יורד

```

#sort by frequency

```

```

freq.df<-freq.df[order(freq.df[,2], decreasing=T),]
head(freq.df,10)

```

###Visualizations###

נתחיל לייצר את ויזואליזציות, אך לפני כן נהפוך את המילה לפקטור כדי ש- ggplot2 יכיר.

###create a word frequency bar plot

#convert the unique words from a string to a factor with unique levels

```
freq.df$word<-factor(freq.df$word, levels=unique(as.character(freq.df$word)))
```

הפונק' בנויה משכבות, בין כל השכבות יש + להפרדה:

- השכבה הראשונה אחראית ליצירת הגרף עם נתוני הצירים של 20 השורות הראשונות. בציר x יהיו המילים ובציר y תהיה השכיחות.

```
ggplot(freq.df[1:20,], aes(x=word,y=frequency))
```

- השכבה השנייה אחראית ליצירת הברים הצבעים, כדי שלא יוצגו סטט' מיותרות נוסף את- "stat="identity".

```
geom_bar(stat="identity", fill='darkred')
```

- השכבה השלישית אחראית לריווח המילים ע"י החלפת הצירים.

```
coord_flip()
```

- השכבה הרביעית אחראית לנראות אחרת (לא חובה להשתמש, אך נראה יותר טוב בעין).

```
theme_gdocs()
```

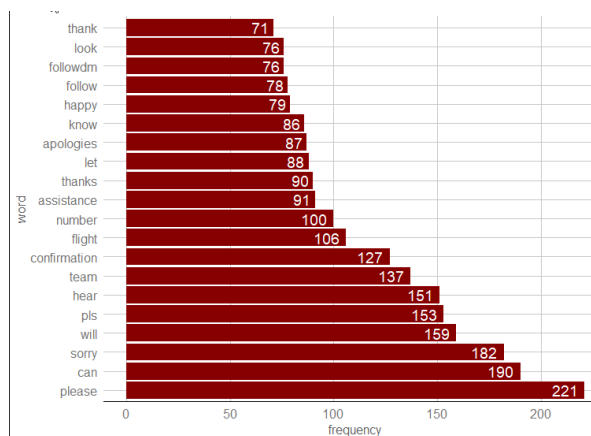
- השכבה החמישית אחראית להוספת מספרים בצורה מדויקת על הברים עצמם, המספרים יופיעו בצבע לבן עם מרחק מסוים.

```
geom_text(aes(label=frequency), colour="white",hjust=1.25,size=5.0)
```

#defining the levels will keep the original order

```
ggplot(freq.df[1:20,], aes(x=word,y=frequency)) + geom_bar(stat="identity", fill='darkred') +
```

```
coord_flip() + theme_gdocs() + geom_text(aes(label=frequency), colour="white",hjust=1.25,size=5.0) #add the numbers on the bars
```



הפונק' findAssocs - אנו רוצים לדעת על מה נציגי השירות של דלתא מתנצלים. ניתן לפונק' את tdm והמילה התנצלות עם ערך סף של 0.11.

word associations - which words appear together with the word 'apologies'

```
associations<-findAssocs(tdm, 'apologies', 0.11) #find the associations for the word 'apologies'. 0.11 is the association threshold
```

קיבלנו רשימה, אנו רוצים להפוך אותה לויז' ברורה. כדי לעשות זאת נהפוך את זה ל- DF.

apologies	issues	appears	deepest	latearriving	you're	work	diligently
delay	0.30	0.25	0.15	0.15	0.15	0.13	0.12
hello	0.12	0.12	0.12	0.12	0.12	0.12	0.12

```
associations<-as.data.frame(associations)
```

ניקח את הכותרות שקיבלנו ב- DF ונהפוך אותם לעמודה. נכניס אותו ל- DF associations\$terms נשתמש בפונק' row.names. לאחר הרצה נראה כי נוספה עמודה חדשה שנקראת terms.

```
associations$terms<-row.names(associations)
```

כדי ש- ggplot2 יעבוד, נהפוך שוב את ה- DF לפקטור.

```
associations$terms<-factor(associations$terms, levels=associations$terms)
#convert to factor
```

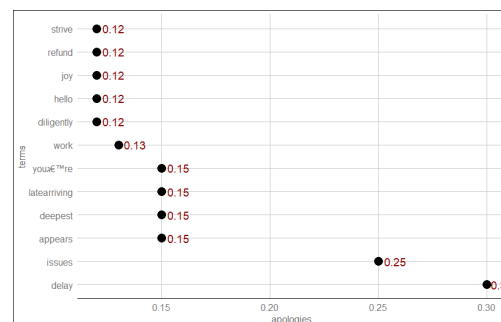
נצייר ויז' באמצעות ggplot.

- השכבה הראשונה נשתמש בנתונים שיש ב- DF. בציר Y נרצה שיופיעו המילים. יופיע הגרף עם המסגרת. ((ggplot(associations, aes(y=terms
- השכבה השנייה אחראית על יצירת הנקודות באמצעות geom_point. בציר x נרצה את עמודת apologies. geom_point(aes(x=apologies), data=associations, size=5
- השכבה השלישית אחראית על נראות התצוגה. (theme_gdocs
- השכבה הרביעית אחראית על הוספת מספור ליד כל נקודה.

```
geom_text(aes(x=apologies,label=apologies),
colour="darkred",hjust=-.25,size=5)
```

scatter plot that shows the association level between the word 'apologies' and the other words

```
ggplot(associations, aes(y=terms)) +
  geom_point(aes(x=apologies), data=associations, size=5) +
  theme_gdocs() +
```



```
geom_text(aes(x=apologies,label=apologies),
colour="darkred",hjust=-.25,size=5)
```

יצירת דינדרוגרמה- בהרצת tdm נגלה שיש כמעט 1700 מילים. אם ננסה לייצר דינדרוגרמה לכמות כזו גדולה של מילים לא נראה כלום על המסך ולא נוכל לתחקר. לכן נצמצם את כמות המילים ונשאיר את המילים היותר נפוצות.

###create a dendrogram - terms will be closer if they appear together in many documents

#how many terms do we have in tdm? how many documents?

tdm

inspect(tdm[1,1:10])

נרצה לצמצם את כמות המילים ולהשאיר רק את המילים הנפוצות. נשתמש בפרמטר שנקרא sparse. נבקש ממנו לכלול מילים שמופיעים בתדירות גבוהה- לפחות 97.5%.
נוסיף את השורה (nrow(tdm2) כדי לדעת כמה מונחים נשארו. נמיר למטריצה כדי לראות תוצאות.

#remove sparse terms. include all terms with 97.5% or less zeros (at least in 34 documents)

tdm2 <- removeSparseTerms(tdm, sparse = 0.975) #42 terms were left

nrow(tdm2)

as.matrix(tdm2[1:42,1:20])

כדי לבנות דינדרוגרמה, הצעד הראשון לבנות מטריצת מרחקים. נשתמש בפונק' dist, כדי לחשב, נשתמש בשיטת חישוב מרחקים, במרחק האוקלידי. ככל שהמס' יותר קטן כך הם יותר קרובים. כלומר המסמכים יותר משותפים, יותר קרובים אחד לשני. כאשר נריץ את השורה ניצור את הדינדרוגרמה.

#dist - computes the distance between the term vectors

#hclust assumes each term is its own cluster and then iteratively attempts to join the two most similar clusters

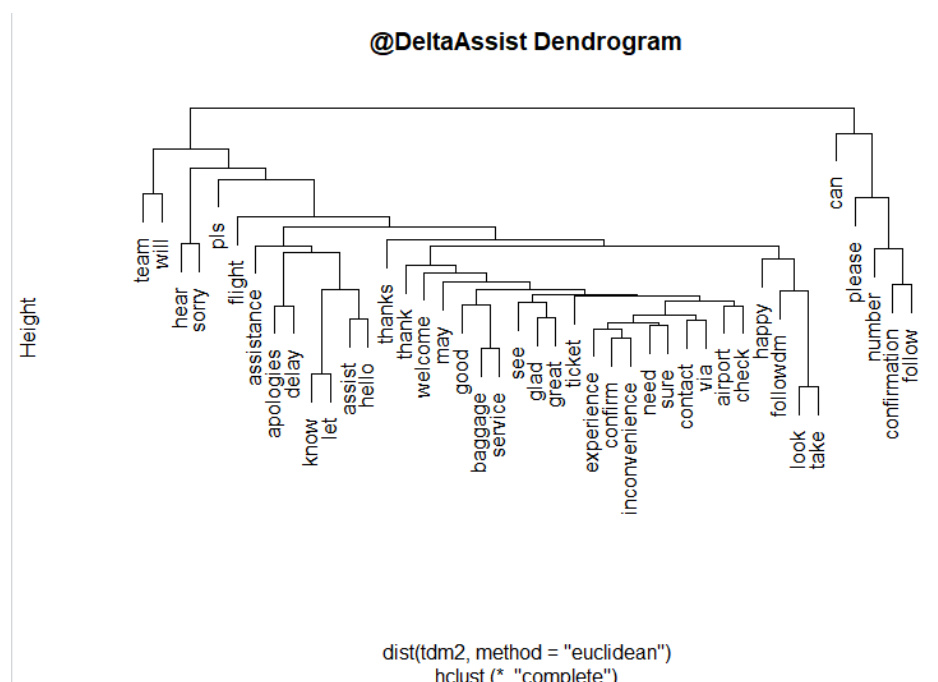
hc <- hclust(dist(tdm2, method = "euclidean"))

כדי לראות את הדינדרוגרמה נריץ את שורת הפלוט. ניתן לה את הכותרת ע"י פונק' main, נוותר על ציר y כי הגובה לא מעניין אותנו במקרה הזה.

#plot the dendrogram

#remove the y axis and add a title

plot(hc,yaxt='n',main = '@DeltaAssist Dendrogram')




```
library(sqldf)
sqldf("select * from 'tweets' where text LIKE '%refund%'")
```

שיעור 5- הכנת נתונים וויזואליזציה 08.04.2022

#Text mining in R

#part 2.2.2 code - commonality/ comparison cloud

setwd("C:\\Users\\lidopo\\OneDrive\\אילן\\כריית\\שולחן העבודה\\בר אילן\\כריית\\שולחן העבודה\\שיעור 5
טקסט\\שיעורים\\שיעור 5")

```
options(stringsAsFactors=FALSE) #don't treat strings as factors (categories)
Sys.setlocale('LC_ALL','C') #English(US)
```

```
library(tm)
library(ggplot2)
library(ggthemes) #has predefined themes and color palettes for ggplot2
visualizations
library(wordcloud)
```

###create a new clean function that works on vectors

הגדרת פונקציית ניקיון בשם clean.vec.

```
clean.vec<-function(text.vec){
  text.vec <- tolower(text.vec)
  text.vec <- removeWords(text.vec, custom.stopwords)
  text.vec <- removePunctuation(text.vec)
  text.vec <- removeNumbers(text.vec)
  text.vec <- stripWhitespace(text.vec)
  return(text.vec)
}
```

####load data

```
amzn <- read.csv('amzn_cs.csv')
delta <- read.csv('oct_delta.csv')
```

```
custom.stopwords <- c(stopwords("english"),'lol','smh','delta','amp')
```

#clean the datasets

הפעלת הפונקציה של clean.vec על הווקטור של הטקסטים שלי. נקבל אוסף של טקסטים שהם נקיים.

```
amzn.vec<-clean.vec(amzn$text)
delta.vec<-clean.vec(delta$text)
head(amzn.vec)
length(amzn.vec) #1045
```

ניקח את כל הציוצים של אמזון ודלתא ונאחד אותם לרשימה אחת גדולה. נשתמש בפונקציה הדבקה שמשרשרת מילה אחרי מילה, בין ציוץ לציוץ תשים רווח " "collapse בסוף לאחר שנשתמש ב length נגלה שיש ציוץ אחד בלבד במקום 1045.

#collapse the vector to a single document

#the object "all" is a corpus with two documents

```
amzn.vec <- paste(amzn.vec, collapse=" ") #paste = Concatenate vectors
after converting to character.
```

collapse - define the separator between the concatenated rows.

another option is to set two colors

```
comparison.cloud(tdm.m, max.words = 50, random.order=FALSE,title.size = 1,
```

```
colors=c('black','darkred'), scale=c(3.5,0.25))
```



על מנת להריץ את Polarized Tag Plot יש להוסיף את הספרייה plotrix.

####construct a Polarized Tag Plot - we would like to see the words with the highest difference in frequency

library(plotrix)

דבר ראשון ניפטר מהמילים שמופיעות רק פעם אחת באחד הקורפוסים. כלומר, נרצה להשתמש אך ורק במילים שמופיעות בשני הקורפוסים.

נשתמש בפונקציה subset, ניתן לו את המטריצה של tdm, נבקש להשאיר רק את המילים שבאמזון יש ערך שגדול מ-0, וגם (&) לגבי דלתא. התוצאה תהיה ממוינת לפי abc. פונקציית tail תציג רק את המילים האחרונות.

כעת נחשב את ההפרש בערך מוחלט ע"י פונקציית abs (absolute value). נשים את הפונקציה difference שנקרא (הפרש).

#only terms that appear in both documents

```
common.words <- subset(tdm.m, tdm.m[,1] > 0 & tdm.m[,2] > 0) #578 terms (out of 4049)
```

```
tail(common.words)
```

```
difference <- abs(common.words[,1] - common.words[,2]) #find the difference between the frequencies
```

add a new column that contains the difference

כעת, נוסיף למילים הנפוצות את עמודת ההפרש ע"י פונקציה cbind. בצפייה על 10 השורות הראשונות נגלה את ההפרש בין אמזון לדלתא.

```
common.words <- cbind(common.words, difference)
```

```
common.words[1:10,1:3]
```

	Amazon	Delta	difference
able	58	34	24
accept	2	7	5
access	14	4	10
account	56	10	46
accurate	2	1	1
actually	1	2	1
add	1	5	4
added	2	3	1
additional	1	6	5
address	13	3	10

נמין לפי ההפרש, שההפרש הכי גדול יהיה ראשון ע"י פונקציית orders לפי העמודה difference.

#order by the third column in decreasing order

```
common.words <- common.words[order(common.words[,3],decreasing = TRUE), ]
```

```
head(common.words)
```

בשביל תצוגה, נבנה DF ל- 25 המילים עם ההפרש הכי גדול נרצה ששם העמודה הראשונה תהיה x והיא תכיל את 25 השורות הראשונות ואת העמודה 1 (אחידה) כחל לכתוב עמודה x ודלתא. כמו כן, תהיה עמודה נוספת של labels- היא תיקח את הש

	x	y	labels
pls	13	153	pls
sorry	303	182	sorry
confirmation	10	127	confirmation
order	113	4	order
number	4	100	number
will	64	159	will
delivery	90	3	delivery
assistance	9	91	assistance
follow	3	78	follow
reach	85	18	reach
apologies	22	87	apologies
like	80	16	like
details	83	20	details
date	63	3	date
touch	62	2	touch
get	82	26	get
thank	15	71	thank
welcome	13	67	welcome

#select the first 25 term values

```
top25.df <- data.frame(x = common.words[1:25, 1], y = common.words[1:25, 2],
```

```
labels = rownames(common.words[1:25, ]))
```

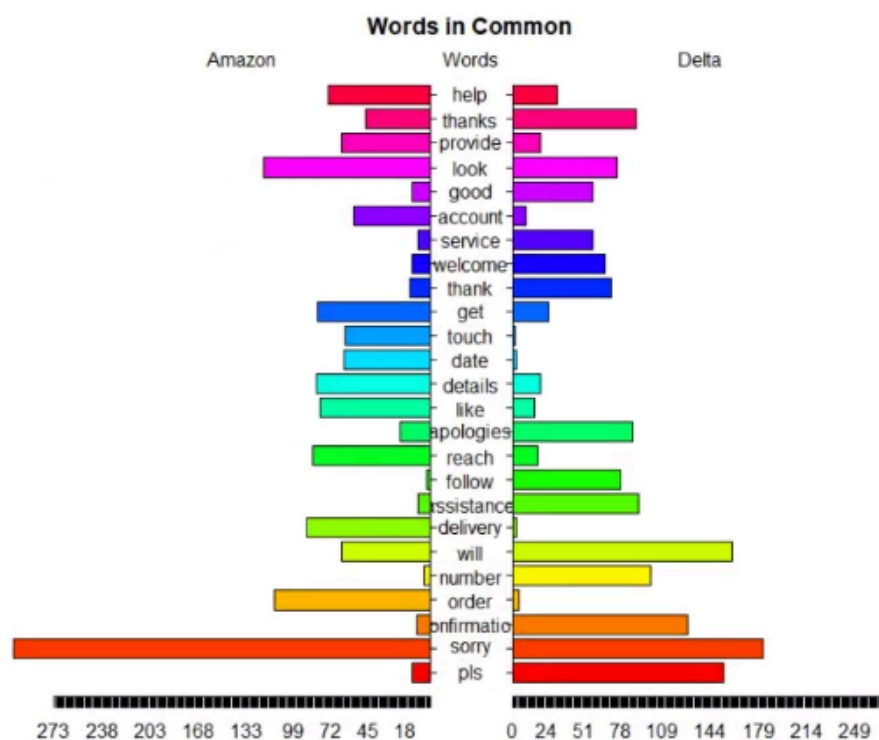
```
head(top25.df)
```

יצירת ה- DF חשובה על מנת לבנות את הפלוט ע"י פונק' pyramid.plot המכילה את עמודה labels, x, y, מרווח (gap), ונוסיף את הכותרות- אמזון, מילים ודלתא. נוסיף גם כותרת ראשית. unit=NULL – במידה ונרצה לראות סוג היחידה

#create the plot

x contains the amazon frequency, y the delta frequency

```
pyramid.plot(top25.df$x, top25.df$y, labels = top25.df$labels,
gap = 30, top.labels = c("Amazon", "Words", "Delta"),
main = "Words in Common", unit = NULL)
```



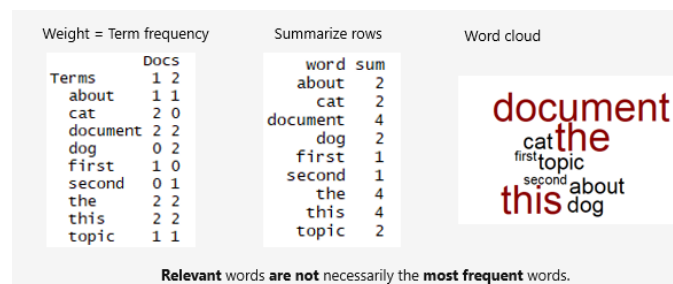
שיעור 6- ניתוח סנטימנט 29.04.2022:
אוגי Bag of Words:

text

first: This document is about a **cat**, the **cat** is the topic of this document

second: This document is about a **dog** the **dog** is the topic of this document

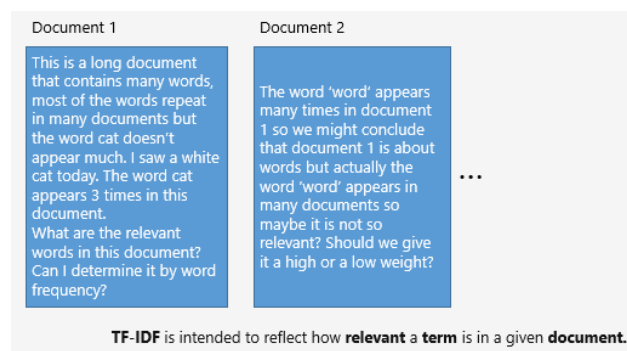
1. Term frequency - סופרים כמה פעמים כל מילה מופיעה בכל מסמך. יוצרים מטריצה ובמטריצה בוחרים משקל TF. לכל מילה רושמים בתאים במטריצה כמה פעמים הוא מופיעה במסמך. המילה document מופיעה 4 פעמים, כנ"ל this, the. אם מילה מסוימת, מופיעה בהרבה מסמכים, כנראה שהיא פחות חשובה. לעומת זאת, המילים חתול וכלב, מופיעים רק במסמך אחד פעמיים, כלומר המילים האלו יחסית נדירים ולכן יש חשיבות. בענן, נראה את המילים הנפוצות ביותר בצבע ואילו את המילים הפחות נפוצות בקטן וללא צבע. המקרה הזה לא מתאים לנו. אנו נרצה לראות את המילים היותר נדירות שחוזרות על עצמן יחסית הרבה.



2. TF-IDF - המטרה - להבליט, לתת משקל גדול יותר למילים שהן חוזרות על עצמן הרבה פעמים במסמך מסוים ומצד שני הן מילים יחסית נדירות בשפה.

1. אוסף גדול של מסמכים, יש מילים שחוזרות על עצמן הרבה פעמים במסמך מסוים ולעומת זאת, אלו מילים שלא חוזרות על עצמן הרבה, בהרבה מסמכים.

2. יש מילים שחוזרות הרבה פעמים במסמך מסוים ומצד שני, חוזרות על עצמן הרבה, בהרבה מסמכים. למשל המילה word, חוזרת על עצמה הרבה פעמים וחוזרת על עצמה בכל המסמכים. לעומת זאת cat, מופיעה במסמך 1-3 פעמים, ובשאר לא מופיעה הרבה.



יש לנו את אותם המשפטים, כאשר נייצר את המטריצה, נקבל משקלים אחרים, המשקל של המילים document, the, ועוד הוא 0. לעומת זאת, המשקל של כלב וחתול יחסית גבוה. מה שבוטל בענן הוא הכלב והחתול.

דוג' נוספת:

```
clean.corpus<-function(corpus){
  corpus <- tm_map(corpus, content_transformer(tolower))
  #corpus <- tm_map(corpus, removeWords, custom.stopwords)
  corpus <- tm_map(corpus, removeNumbers)
  corpus <- tm_map(corpus, removePunctuation)
  corpus <- tm_map(corpus, stripWhitespace)
```



```

return(corpus)
}

```

נייבא את הקובץ, ניצור את הקובץ ורק בשורה התחתונה, נשתמש בניקיון על המשתנה `.corpus`.

```

text.df<-read.csv('test.tf.idf.csv') #read the file
text<-data.frame(doc_id=seq(1:nrow(text.df)),text=text.df$text) #create a
dataframe of text and doc_id
#custom.stopwords <- c(stopwords("english")) #words to remove
corpus <- VCorpus(DataframeSource(text)) #create a corpus
corpus<-clean.corpus(corpus) #apply the cleaning function

```

נייצר TM עם TF, ניצור מטריצה וכך נוכל לראות כמה פעמים כל מילה, מופיעה בכל המסמכים.

```

#create Term Document Matrix
##### weight by term frequency
tdm<-TermDocumentMatrix(corpus, control=list(weighting=weightTf))
#weightTf weights a TDM by term frequency
inspect(tdm[1:9, 1:2]) #inspect the result
tdm.m<-as.matrix(tdm) #convert to a matrix
term.freq<-rowSums(tdm.m) #summarize the rows (number of appearances of
each term)
term.freq # term frequency

```

נהפוך את הווקטור ל- DF המכיל את המילה וכמה פעמים היא מופיעה במסמכים, העמודה הראשונה תהיה `word`, העמודה השנייה תהיה `sum`. נמין לפי גודל השכיחות בשביל סדר בעיניים. ניצור ענן, נפעיל את הפונקציה לפי העמודות שיצרנו קודם, נוסיף מגבלות של מיני' ומקסי' וצבעים.

```

freq.df<-data.frame(word=names(term.freq),sum=term.freq) #create a
dataframe with two columns, word and sum
freq.df<-freq.df[order(freq.df[,2], decreasing=T),] #sort by frequency
freq.df
wordcloud(freq.df$word,freq.df$sum, max.words = 10,min.freq=1,
colors=c('black','darkred'))

```

ההבדל הוא רק ביצירת המשקל $TF \cdot IDF$ במקום TF בקוד למעלה.

```

#####
# TF-IDF weight example
tdm.tfidf<-TermDocumentMatrix(corpus, control=list(weighting=weightTfIdf))
#weightTfIdf weights a TDM by TF-IDF
inspect(round(tdm.tfidf[1:9, 1:2],2)) #inspect the result
tdm.m<-as.matrix(tdm.tfidf) #convert to a matrix
term.freq<-rowSums(tdm.m) #summarize the rows
term.freq[1:9] # term weight

```

```

freq.df<-data.frame(word=names(term.freq),sum=round(term.freq,2)) #create
a dataframe with two columns, word and sum
freq.df
wordcloud(freq.df$word,freq.df$sum, max.words = 10,min.freq=1,
colors=c('black','darkred'))

```

כדי להדגים את המשקל הבינארי, צריך לטעון מסמך BIN, קובץ CSV. באותה צורה, ניצור DF שמכיל doc_id.

```
#####  
#####binary weight example  
#####first, lets start from the default term frequency  
#read the file  
text.bin.df<-read.csv('test.bin.csv')  
#create a dataframe of text and doc_id  
text.bin<-data.frame(doc_id=seq(1:nrow(text.bin.df)),text=text.bin.df$text)  
#create a corpus  
corpus.bin <- VCorpus(DataframeSource(text.bin))  
#clean the corpus  
corpus.bin<-clean.corpus(corpus.bin)
```

אנו רוצים לראות את המקרה הרגיל, איך הענן נראה עם משקל TF.

```
tdm.bin<-TermDocumentMatrix(corpus.bin, control=list(weighting=weightTf))  
#weightTF - weight by term frequency  
inspect(tdm.bin[1:9, 1:2]) #inspect the result  
tdm.m<-as.matrix(tdm.bin) #convert to a matrix  
term.freq<-rowSums(tdm.m) #summarize the rows  
term.freq[1:9] # term weight
```

```
freq.df<-data.frame(word=names(term.freq),sum=round(term.freq,2)) #create  
a dataframe with two columns, word and sum  
freq.df  
wordcloud(freq.df$word,freq.df$sum, max.words = 10,min.freq=1,  
colors=c('black','darkred'))
```

אם נרצה להשתמש את המשקל הבינארי, נוסיף אותו לקוד במקום TF.

```
#####binary weight  
#changing to Binary weight - 1 if a word appears in a document, 0 otherwise  
tdm.bin<-TermDocumentMatrix(corpus.bin, control=list(weighting=weightBin))  
#weightBin - Binary weight  
inspect(tdm.bin[1:9, 1:2]) #inspect the result  
tdm.m<-as.matrix(tdm.bin) #convert to a matrix  
term.freq<-rowSums(tdm.m) #summarize the rows  
term.freq # term weight
```

```
freq.df<-data.frame(word=names(term.freq),sum=term.freq) #create a  
dataframe with two columns, word and sum  
freq.df  
wordcloud(freq.df$word,freq.df$sum, max.words = 10,min.freq=1,  
colors=c('black','darkred'))
```

נרצה להריץ את המשקל TF-IDF על המסמך החדש. ניצור את ה-TM עם TF-IDF. חתול קיבל משקל יותר גדול מכלב מכיוון ובמסמך היו פחות מילים ולכן המשקל של הכלב נמוך מהמשקל של החתול.

```
#####TF-IDF weight  
tdm.bin<-TermDocumentMatrix(corpus.bin,  
control=list(weighting=weightTfIdf)) #weight TF-IDF
```

```
inspect(round(tdm.bin[1:9, 1:2],4)) #inspect the result
tdm.m<-as.matrix(tdm.bin) #convert to a matrix
term.freq<-rowSums(tdm.m) #summarize the rows
term.freq # term weight
```

```
freq.df<-data.frame(word=names(term.freq),sum=round(term.freq,2)) #create
a dataframe with two columns, word and sum
freq.df
wordcloud(freq.df$word,freq.df$sum,max.words = 10,min.freq=1,
colors=c('black','darkred'))
```

how come cat is bigger than dog?

שיעור 7- ניתוח סנטימנט (רגש בטקסט) :06.05.2022

יש צורך לנתח רגשות חיוביים ושליילים על מנת להבין האם הטקסט הוא חיובי או שלילי. המטרה- למצוא רגשות בתוך הטקסט ולנתח את הטקסט. קשה לזהות האם הטקסט חיובי / שלילי בגלל ציניות, סלנג מקומי, יכול להיות שנכתוב ביקורת חיובית ושליילית על אותו המקום. אנחנו לא נרצה לסווג את הטקסט לרגש ספציפי, אלא למצוא האם הטקסט חיובי / ניטרלי / שלילי.

שימושים- ניתוח סקרים, פוסטים על מוצרים / אנשים כדי לראות איך הרגש משתנה לאורך זמן.

לקסיקון סובייקטיביות- רשימה של מילים הקשורות למצב קוטביות (או רגש) מסוים. לקסיקונים בנויים בצורה איכותית, יש הרבה לקסיקונים בחוץ, רוב הלקסיקונים מכילים 6000-8000 מונחים.

לכל מילה נרצה לדעת האם יש לה סממן חיובי או שלילי, נראה זאת בעמודה polarity. לעמודה part of speech יש משמעות. האם המילה היא פועל, שם עצם, שם תואר וכו'...

Type	Length	Word	Part of Speech	Stemmed y/n	Polarity
Weak	1	Abundant	Adj	N	Positive
Weak	1	Abundance	Noun	N	Positive
Strong	1	Accede	Verb	Y	Positive
Weak	1	Accept	Verb	Y	Positive

קוטביות- ניקוד סנטימנט פשוט ע"י שיטת חיבור וחיסור.

Example: "Sentiment analysis in R is good yet challenging"

נחפש בכל המילים, האם המילים נמצאים במילון שלי. המילה good מופיעה ויש לה ציון +1, לעומת זאת ל- challenging יש ציון -1 ולכן הסכום של שתי המילים הוא 0.

Word	Score
Good	+1
Challenging	-1

מחליפי תוקף Valence shifters:

יש שלושה סוגי מחליפים- הופך כיוון (negator), מחליף תוקף שמחזק (amplifier) ומחליף תוקף שמחליש (de-amplifier).

I do not like herring

I really like mango

I hardly like to drive

במשפט - I do not like herring יש לנו את המילה not לפני המילה like. ברגע שיש לנו מקרה כזה, במקום לקבל ציון של +1 נקבל ציון של -1.

לעומת זאת, המילה really מחזקת, במקרה הזה הציון של +1 יקבל ציון של +1.5.

המילה *hardly*, לא הופכת את המשפט, אך היא מחלישה את המילה *like*.
 דוגמאות מתוך ספריית qdap. אסור להשתמש ב- stopwords לפני שמחשבים את הציון של הטקסט. נעשה ניקיונות שהם טובים לפני ניתוח הרגש, לאחר מכן כדי ליצור ענן מילים נשתמש בשאר הניקיונות.

Valence shifters in 'qdap'

Negator

```
> qdapDictionaries::negation.words
```

[1] "ain't"	"aren't"	"can't"	"couldn't"	"didn't"	"doesn't"	"don't"	"hasn't"	"isn't"
[10] "mightn't"	"mustn't"	"neither"	"never"	"no"	"nobody"	"nor"	"not"	"shan't"
[19] "shouldn't"	"wasn't"	"weren't"	"won't"	"wouldn't"				

Amplifier (intensifier)

```
> qdapDictionaries::amplification.words
```

[1] "acute"	"acutely"	"certain"	"certainly"	"colossal"	"colossally"	"deep"
[8] "deeply"	"definite"	"definitely"	"enormous"	"enormously"	"extreme"	"extremely"
[15] "great"	"greatly"	"heavily"	"heavy"	"high"	"highly"	"huge"
[22] "hugely"	"immense"	"immensely"	"incalculable"	"incalculably"	"massive"	"massively"
[29] "more"	"particular"	"particularly"	"purpose"	"purposely"	"quite"	"real"
[36] "really"	"serious"	"seriously"	"severe"	"severely"	"significant"	"significantly"
[43] "sure"	"surely"	"true"	"truly"	"vast"	"vastly"	"very"

De-amplifier (downtoner)

```
> qdapDictionaries::deamplification.words
```

[1] "barely"	"faintly"	"few"	"hardly"	"little"	"only"	"rarely"
[8] "seldom"	"slightly"	"sparsely"	"sporadically"	"very few"	"very little"	

חישוב פונקציית הרגש האלגוריתם עובר על כל המילים במשפט ומחשב מילים שיש לו במילון. המילה good נמצאת במילון ולכן היא מקבלת +1. האלגוריתם מגדיר קלאסטר. אשכול של מילים- 4 מילים לפני המילה ועוד 2 אחרי. במקרה שלנו אין מילים אחרי המילה, אלא רק נקודה ולכן לא נוריד סימני פיסוק. נחפש האם בקלאסטר יש negator, amplifier או de-amplifier. המילה very מחזקת ולכן יתווסף ציון של +0.8 לנוסחה. במשפט יש 7 מילים, לבסוף ננרמל אותן כדי לחשב את הציון הסופי. אם היה לנו negator, זה לא היה משנה את הסכום, אלא רק משנה את התוצאה הסופית מפלוס למינוס. אם היה de-amplifier זה היה מחליש את הציון.

Amplifier "very" = +0.8

+

Positive Polarity "good" = +1.0

=

1.8

Positive word cluster

S1 = "Sentiment analysis in R is very **good**."

7 words in total

Polarity(S1) = $\frac{1.8}{\sqrt{7}} = 0.68$

```
> polarity('the food is very good')
all total.sentences total.words ave.polarity
1 all 1 5 0.805
> polarity('the food is not good')
all total.sentences total.words ave.polarity
1 all 1 5 -0.447
```

Class Exercise:

- Find the polarity of the following sentences, use both 'qdap' and 'sentimentr' packages (polarity and sentiment functions) –
 - "This is a great day"
 - "This is a great day, lol"
 - "This restaurant is awful. The food is really bad."
 - "This restaurant is awful the food is really bad"

- "Here is my classroom"
2. What is the default 'sentimentr' dictionary (lexicon)? Print 20 values of this dictionary.
 3. Print the "sentiment" function arguments (use the function 'args'). What is the default value of "amplifier.weight" argument?

Answers:

Q1:

```
library(qdap)
library(sentimentr)
```

```
polarity("this is a great day")
polarity("This is a great day, lol")
polarity("This restaurant is awful. The food is really bad")
polarity("This restaurant is awful the food is really bad")
polarity("Here is my classroom")
```

Q2:

```
head(lexicon::hash_sentiment_jockers_rinker, 20)
```

Q3:

```
args("sentiment")
```

אם נרצה להוסיף מילים נוספות למילון, נשתמש בפונקציה 'update_polarity_table' נשים את המילים במרכאות ונגדיר את הציון בווקטור. לפני השימוש במילים החדשות, הציון היה חיובי ולאחר מכן שלילי.

```
library(lexicon)

updated_hash_sentiment <-
  sentimentr::update_polarity_table(lexicon::hash_sentiment_jockers_rinker,
    x = data.frame(words = c('be cleaner', 'repairs'), polarity = c(-1, -1),
      stringsAsFactors = FALSE)
  )

> sentiment('This apartment can be cleaner')
  element_id sentence_id word_count sentiment
1:         1           1           5 0.2236068

> sentiment('This apartment can be cleaner', polarity_dt=updated_hash_sentiment)
  element_id sentence_id word_count sentiment
1:         1           1           5 -0.4472136
```

חישוב קוטביות מערך הנתונים 'sentimentr' - אם נרצה לחשב סנטימנט ממוצע לכל המשפט, נשתמש בפונקציה 'sentiment by'.

```
#calculate the polarity of each sentence

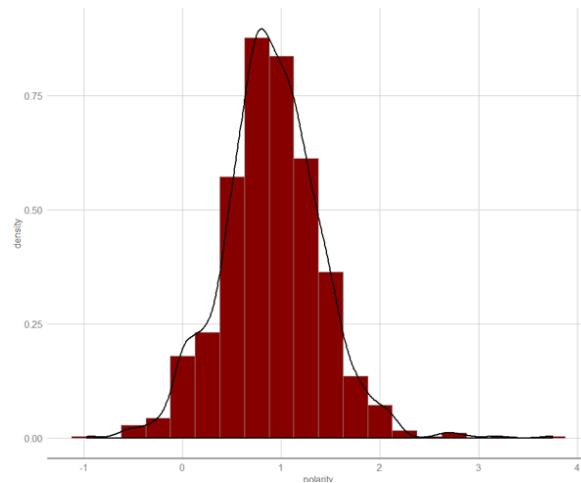
bos.pol<-sentiment(bos.airbnb$comments)

#calculate the polarity of each comment

bos.pol<-sentiment by(bos.airbnb$comments)
```

ענן מילים סנטימנטים - נרצה להבחין בין דירה עם ציונים חיוביים לבין דירה עם ציונים שליליים. נייצר קורפוס שמכיל מסמך אחד שמכיל את כל הביקורות החיוביות והשני עם ביקורות שליליות. יראה באילו מילים ייחודיות משתמשים לפוסטים חיוביים לעומת שליליים ואילו מילים אחרות משותפות, ובכך יש סנטימנט מעורב.

חלוקת קוטביות של ביקורות Airbnb - נייצר היסטוגרמה ונסתכל על ההתפלגות של ציוני הסנטימנט. ניתן לראות היכן נמצא הריכוז של הביקורות שלנו. מחשב את הקוטביות של כל הערה, יוצר היסטוגרמה ותרשים צפיפות כדי לראות את התפלגות הקוטביות.



שיעור 8- ניתוח סנטימנט (רגש בטקסט) 13.05.2022:

#part 3.2 code Sentiment Scoring

```
setwd("C:\\Users\\idopo\\OneDrive\\אילן כריית\\שולחן העבודה\\בר אילן\\כריית")
```

```
options(stringsAsFactors=FALSE)
```

###polarity calculation

###simple example

sentiment('wow, this is an excellent restaurant')

sentiment('this is a bad restaurant')

comma

sentiment('this is a very good restaurant I like it') #no comma, 'very' affects 'like'

```
# n.before
```

```
sentiment('this is a very good restaurant and I like it') # 'very' affects 'like'
(n.before = 5)
```

question

```
sentiment('do you like it?', question.weight = 0) # assumption - pure questions  
are not polarized
```

like - non-verb form

במקרה של משפט משולב, נקבל שני אלמנטים עם שני ציונים. נפעיל את `sentence_by` כדי לקבל ציון אחד על כל המשפטים. נקבל ממוצע של החיובי והשלילי. בחלקו התחתון נוסיף פרמטר `averaging.function`. כשנחשב את הממוצע, תיתן משקל יותר גדול למשקלים שליליים.

sentiment vs. sentiment by

```

sentiment('the apartment was dirty, I hardly could sleep at night because of
the noise. Although I really liked the kitchen.')
sentiment_by('the apartment was dirty, I hardly could sleep at night because
of the noise. Although I really liked the kitchen.') #the average
sentiment_by('the apartment was dirty, I hardly could sleep at night because
of the noise.
Although I really liked the kitchen.', averaging.function =
average_weighted_mixed_sentiment) #the affective state is negative.
פונקציה זו נשתמש על מנת למצוא אילו מילים הוא מצא במילון.
extract_sentiment_terms('the apartment was dirty, I hardly could sleep at night
because of the noise. Although I really liked the kitchen.')

```

נרכז את כל המילים החיוביים ושליילים בווקטור. נפעיל את הפונק' get_sentences שתפריד בין המשפטים. נפעיל את הפונק' sentiment על מנת לקבל תוצאות חיוביות / שליליות. נקבל שהמשפט הראשון היה שלילי, השני חיובי והשלישי ניטרלי. נהפוך את הסנטימנט ל-DF, המשפטים יהיו מופרדים ונוסיף עמודה של סנטימנט

```

### split to sentences - if we want to analyze each sentence separately
text <- c('the apartment was dirty, I hardly could sleep at night because of the
noise. Although I really liked the kitchen.','another text')
textSentences <- get_sentences(text)
sent <- sentiment(textSentences)
# convert to separate sentences
sent.df <- data.frame(text = unlist(textSentences), sent)

```

	text	element_id	sentence_id	word_count	sentiment
1	the apartment was dirty, I hardly could sleep at night because of the noise.	1	1	14	-0.2538982
2	Although I really liked the kitchen.	1	2	6	0.6225786
3	another text	2	1	2	0.0000000

הוספת מילים ללקסיקון:

```
###adding words to the polarity lexicon
```

הלקסיקון הקלאסי נקרא jockers_rinker.

```
#check if a word exists in the lexicon
```

```
lexicon::hash_sentiment_jockers_rinker[1:30] #different weights
```

```
# the word "hairy" weight
```

נבדוק אם המילה hairy נמצאת במילון והיא מקבלת ציון של -0.1. לאחר מכן נבדוק שוב על מילון אחר שנקרא huliu ונקבל NA כלומר שזה לא קיים. נבדוק זאת שוב על מילים נוספות ונבדוק האם הן קיימות.

```
lexicon::hash_sentiment_jockers_rinker["hairy"]
```

```
lexicon::hash_sentiment_huliu["hairy"]
```

```
lexicon::hash_sentiment_jockers_rinker["be cleaner"]
```

```
lexicon::hash_sentiment_jockers_rinker["repair"]
```

נניבא ספרייה שנקראת לקסיקון, נפנה לפונק' שנקראת update_polarity_table ניתן לה את המילון ונוסיף את המילים 'repairs', 'be cleaner'. נגדיר אותן עם ציון -1.

```
# add new words to the dictionary
```

```
library(lexicon)
```

```
updated_hash_sentiment <-
```

```
sentimentr::update_polarity_table(lexicon::hash_sentiment_jockers_rinker,
```



```

x = data.frame(
  words = c('be cleaner', 'repairs'),
  polarity = c(-1, -1),
  stringsAsFactors = FALSE
)

```

לאחר מכן נריץ את המילים כדי לראות את התוצאה. בשורה הראשונה נקבל ציון חיובי כי לא הגדרנו את המשפט בלקסיקון שאליו הוספנו את המילה ואילו במשפט השני כן.

```

sentiment('This apartment can be cleaner')
sentiment('This apartment can be cleaner',polarity_dt=updated_hash_sentiment)

```

מחליפי תוקף איך אנחנו רואים אילו מילים מחליפות תוקף. למילון יש עמודת מילה ועמודת קוד. בעזרה ניתן לראות ש- 1 זה 2 negator, 2 amplifiers, 3 de-amplifiers.

```

#valence shifters
lexicon::hash_valence_shifters
#negators
lexicon::hash_valence_shifters[y == 1]
#amplifiers [intensifiers]
lexicon::hash_valence_shifters[y == 2]
#de-amplifiers [downtoners]
lexicon::hash_valence_shifters[y == 3]

```

ניצור את הרגש עצמו. כלומר, האם יש רגש שמח / עצוב וכו'... ניצור וקטור עם משפטים. ככל שמתקדמים המשפטים הופכים ליותר שליליים. הבעיה, הוא לא מבין ש- not happy זה שלילי בניגוד ל- unhappy.

```

# emotion terms
text <- c(
  "I love it",
  "I'm very happy right now",
  "I am not afraid of you",
  "I'm not angry with you",
  "I went to the terrible store",
  "I hate it when you lie to me, it's so humiliating",
  "I'm not happy anymore, it's time to end it",
  "I'm unhappy, want to die",
  "He is killing me")

```

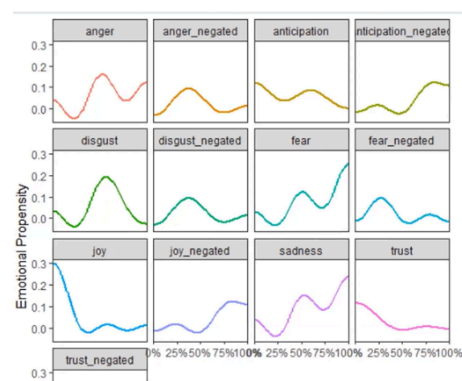
נריץ פונק' שנקראת `extract_emotion_terms`. ניתן לה את הטקסטים, נשתמש במילון שנקרא `hash_nrc_emotions` כדי להתייחס לקידומת un כשלילית.

element_id	sentence_id	anger	anticipation	disgust	fear	joy	sadness	trust	surprise
1:	1					love			<NA>
2:	2		happy			happy		happy	<NA>
3:	3				afraid				<NA>
4:	4	angry							<NA>
5:	5	terrible	store	terrible	terrible		terrible		<NA>
6:	6	hate, lie		hate, humiliating, lie	hate		hate, lie		<NA>
7:	7		happy, time			happy		happy	<NA>
8:	8	unhappy		unhappy	die		die, unhappy		<NA>
9:	9	killing			killing		killing		<NA>

```

# emotion terms
extract_emotion_terms(text, emotion_dt =
lexicon::hash_nrc_emotions, un.as.negation
= TRUE)

```



לאחר הרצת התרשים, ניתן לראות את השינויים ברגשות.

```
plot(emotion(text))
```

###Airbnb case study

טעינת הספריות והבאת הקובץ, לא נעשה ניקיונות לפני.

#bos_airbnb_1k.csv contains 1000 Airbnb reviews related to a particular stay at an apartment or house in Boston.

```
library(ggplot2)
```

```
library(ggthemes) #has predefined themes and color palettes for ggplot2 visualizations
```

```
library(sentimentr)
```

```
bos.airbnb<-read.csv('bos_airbnb_1k.csv')
```

הפעלת פונק' sentiment by על עמודת התגובות ונדפיס את 6 השורות הראשונות.

```
#calculate the polarity of each comment
```

```
bos.pol.sentimentr<-sentiment_by(bos.airbnb$comments)
```

```
head(bos.pol.sentimentr) #ave_sentiment is the average polarity score
```

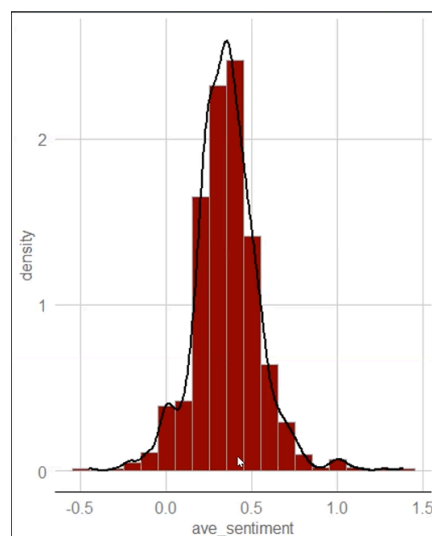
כדי לבדוק את ההתפלגות של הסנטימנט, נייצר היסטוגרמה עם הנתונים. נקבל שרוב ההתפלגויות חיוביות באיזור 0.4.

```
#create a histogram and a density plot to see the polarity distribution
```

```
ggplot(bos.pol.sentimentr, aes(x=ave_sentiment, y=..density..))+
```

```
theme_gdocs() + #..density..is a computed variable, used to calculate the density of x, part of ggplot2.
```

```
geom_histogram(binwidth=.1,fill="darkred",colour="grey60", size=.2) +  
geom_density(size=.75)
```



נוכל לבצע scale הממרכז את ציוני הסנטימנט ע"י סטנדרטיזציה בכך שהממוצע יהיה 0. ניצור DF חדש שנקרא bos.airbnb\$sentiment_scaled המייצרת עמודה חדשה שנכניס לתוכה את הערך המנורמל. נסיף עמודה bos.airbnb\$sentiment המכילה את הציון המקורי.

#the center is 0.4

#scaling the polarity score vector moves the average to zero

#add the polarity to the dataset

```
bos.airbnb$sentiment_scaled<-scale(bos.pol.sentimentr$ave_sentiment)
bos.airbnb$sentiment<-bos.pol.sentimentr$ave_sentiment #not scaled
```

נבדוק כמה ביקורות חיוביות, ניטרליות ושליליות יש לנו. נפעיל את הפונק' table שבתוכה נפעיל את הפונק' sign. בהרצת הפונקציה המנורמלת, נקבל 509 תוצאות שליליות ו- 491 חיוביות. בהרצת הפונקציה הרגילה, נקבל 28 תוצאות שליליות, 17 ניטרליות ו- 955 חיוביות. זה אומר שבביקורת החיוביות מתחבאים ביקורות שליליות.

How many positive and negative reviews

```
table(sign(bos.airbnb$sentiment_scaled)) #scaled #sign returns -1,0 or 1
```

```
table(sign(bos.airbnb$sentiment)) #not scaled
```

```
bos.airbnb[bos.airbnb$sentiment < 0, "comments"]
```

עלינו למצוא את התגובות שקטנות מהממוצע ולכן נשמור את הביקורות כקובץ csv. בעמודת sentiment_scaled (CO) נבחר תוצאות קטנות מ- 0 ואילו ב- sentiment (CP) נבחר תוצאות גדולות מ- 0 כדי לבדוק האם הגיוני שיש הבדל. בשורה התחתונה, ניקח רק מי שהסנטימנט שלו קטן מ- 0 בעמודת התגובות. בתגובה המצורפת ניתן לראות שהביקורת עצמה חיובית, אך עדיין הציון המנורמל שלילי ואילו הציון הרגיל חיובי.

```
I had a great time at Christine's.
The place was spacious, well decorated and as noted earlier, perfectly located.

The communication and check in/out instructions were effective.

I'd absolutely recommend Christine's place and stay again, if she'd have me.
```

#write the new dataset with the polarity to a csv file

```
write.csv(bos.airbnb,"bos.airbnb with polarity - compare scale and sentimentr.csv")
```

שיעור 9- למידה לא מופקחת – 27.05.2022 Topic Modeling

המטרה- לקחת אוסף גדול של מסמכים (טקסטים) ולחלק אותם לנושאים. למשל, ניתוח אשכולות. כל מסמך יכול להיות בנוי מכמה נושאים. למשל, 90% מהמסמכים נמצאים בנושא 1, 10% מהמסמכים, נמצאים בנושא 2. תיאור כל נושא לפי "מילות מפתח". השימושים- מערכות ממליצים. למשל, המלצה על כתבות דומות. מציאת שאלות הדומות זו לזו. הקצאת קו"ח לפי סוגים שונים.

LDA- הקצאת דיריכלט סמויה- מציאת הסתברויות לפי התפלגויות:

- סמוי- מתייחס לכל מה שאיננו יודעים ומוסתר בנתונים. הנושאים שמהם מורכב המסמך אינם ידועים.
- דיריכלט- "הפצת הפצות", הפצת נושאים במסמכים והפצת מילים בנושא.
- הקצאה- לאחר שתהיה לנו דיריכלה, נקצה נושאים למסמכים ומילות המסמך לנושאים.

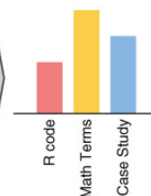
Each document is constructed from multiple topics

You can use the following code to execute a **Spherical K-Means** approach on the **example HR** texts. Remember the goal is to identify specific terms, attributes or **work experiences** that can help **educate HR recruiters at your call center**. You hope the new approach yields more insights and a possible **recommendation for your recruiting team**.

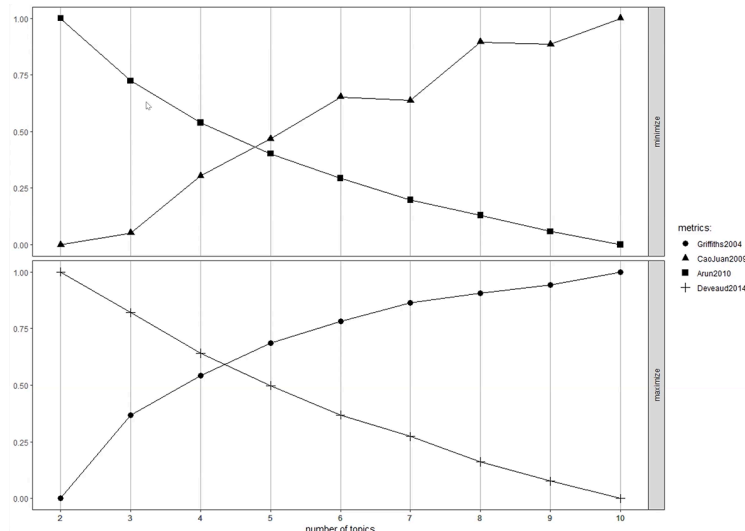
Start by loading the **Spherical K-Means** and **cluster ensemble** libraries in addition to the previous section's libraries. To get started the code below skips ahead past corpus creation and cleaning to recreate the **work experiences** DTM that has TFIDF weighting as shown in the previous **K-Means** section.

```
library(skmeans)
library(clue)
wk.dtm<-DocumentTermMatrix(wk.corpus, control=list(weighting=
weightTfIdf))
```

Next you will actually perform the **Spherical K-Means clustering**. To do so you evoke the **skmeans** function. It is applied to the **work experience** DTM called **wk.dtm**. The next parameter specifies the **number of clusters** similar to simple **K-Means**. Once again you decide to choose three clusters. The next parameter is "m". This parameter is the fuzziness of the clusters as the **algorithm** is constructed. This allows the **cluster** sphere have a border that is not concretely defined. This is analogous to a border between two countries that have a demilitarized zone in between. Both countries could claim this fuzzy shared boarder. The m parameters ranges from 1 upward. A value of 1 would mean to perform a hard partition without any "fuzziness" between cluster boarders. The higher the m value the fuzzier the boarders. Here the m parameter is 1.2 meaning some fuzziness is needed. The last section of



כיצד לקבוע מס' נושאים? תלוי באפליקציה ובקובץ הנתונים. לפעמים נרצה לחלק את המסמכים לקבוצות קטנות או לחילופין, נרצה לעבור על המסמכים ולכן נחלק לכמה קבוצות בודדות. ישנם 4 מדדים לקביעת כמות הקבוצות (מופיע בצילום מסך מטה). הביקורת על השיטה- חוסר יציבות- בכל הרצה נקבל תוצאות שונות, אלגוריתמים שונים יפיקו נושאים שונים.



#Text mining in R

#part 4.1 code Topic modeling

שולחן העבודה\בר אילן\כריית\OneDrive\\
 (טקסט\שיעורים\שיעור 9)

הספרייה שיש בה את LDA נקראת topicmodels.

```
library(topicmodels)
library(tm)
library(rtweet)
library(wordcloud)
library(textstem)
options(stringsAsFactors = F)
Sys.setlocale('LC_ALL','C')
```

#already done

```
#tweets <- search_tweets(q = "#israel", n = 1000, lang = "en", type =
"popular", include_rts = FALSE)
#tweets$text <- iconv(tweets$text, "latin1", "ASCII", sub="") #convert Latin to
ASCII
#save_as_csv(tweets, file_name = "israelTweetsPop25052022.csv",
prepend_ids = TRUE, na = "",
# fileEncoding = "UTF-8")
```

tweets file

```
tweets <- read.csv("israelTweetsPop25052022.csv")
```

```
class(tweets)
head(tweets$text, 5)
```

הורדת קישורים, amp/ (אמפרסנד) והכנת רשימת stopwords.

#remove links

```
tweets$text <- gsub("http\\S*", " ", tweets$text)
tweets$text <- gsub("\\n", " ", tweets$text)
tweets$text <- gsub("&", " ", tweets$text)
```

custom.stopwords

```
c(stopwords("english"), 'rt', 'israel', 'israels', 'amp', 'israeli', 'israelis')
```

<-

הגדרת פונק' ניקיון- הפיכה לקורפוס, מעבר לאותיות קטנות, הסרת stopwords, הסרת מס', הסרת סימני פיסוק, צמצום רווחים מיותרים והפעלת סטמינג (ניתן לוותר עליו אם הוא מפריע).

```
#clean.data function
clean.data <- function(text){
  myCorpus <- VCorpus(VectorSource(text))
  myCorpus <- tm_map(myCorpus, content_transformer(tolower)) #convert to lower case
  myCorpus <- tm_map(myCorpus, removeWords, custom.stopwords)
  #remove stopwords
  myCorpus <- tm_map(myCorpus, removeNumbers) #remove numbers
  myCorpus <- tm_map(myCorpus, removePunctuation) #remove punctuation
  myCorpus <- tm_map(myCorpus, stripWhitespace) #remove extra whitespace
  myCorpus <- tm_map(myCorpus, stemDocument)
  return(myCorpus)
}
tweetCorpus <- clean.data(tweets$text)
for(i in 1:5){
  print(tweetCorpus[[i]][1])
}
```

קטעי קוד העוזרים לניתוח הדברים- מציאת המילים הנפוצות בטקסט.

```
# find top terms
# TDM, *without* tfXidf weighting
#tdm <- TermDocumentMatrix(tweetCorpus)
#inspect(tdm[1:15, 1:15])

# view top frequent terms
#term.freq <- rowSums(as.matrix(tdm)) #summarize each row (number of appearances of each term)
#top.terms <- term.freq[order(term.freq, decreasing = T)][1:100]
#top.terms[1:10]
```

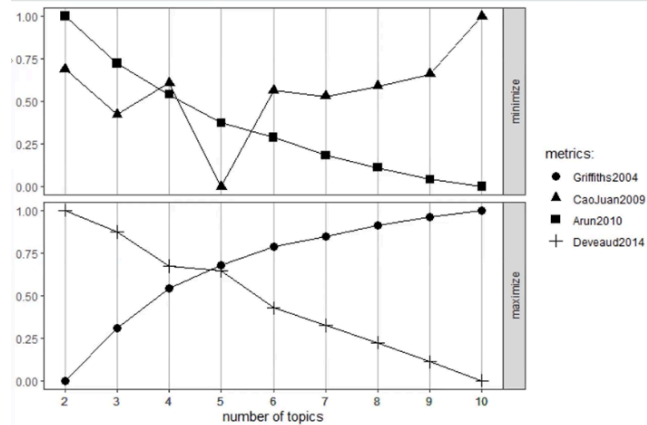
לפני שמריצים את topic modeling צריך לבנות DTM. לא נשתמש במשקל של TF-IDF. כלומר, כל תא במטריצה, יכיל כמה פעמים מילה מסוימת תופיע במסמך מסוים. שורת הקוד האחרונה היא צמצום מילים שמופיעות מעט מאוד פעמים. נרצה להשאיר מילים שמופיעות יותר מפעם אחת.

```
## topic modeling
# DTM, *without* tfXidf weighting
dtm <- DocumentTermMatrix(tweetCorpus)
# we can remove sparse terms
#dtm <- removeSparseTerms(dtm, .979)
```

בחירה לכמה נושאים לחלק. כדי לדעת כמה מדדים מוצעים לנו נשתמש בספריה ldatuning. נריץ פונק' שנקראת FindTopicsNumber שתריץ בין נושא 1 ל- 10 עם קפיצות של 1 (אם יש הרבה מסמכים אפשר לדלג עליהם), נבחר איזה מדדים (metrics) ע"י וקטור, נבחר מתודה (דברים ש- R מציע). נקבע את הסיד כדי לקבל תוצאה זהה ונגדיר את אלפא כמס' קטן = 0.1. כלומר, שמסמך מסוים ישתייך לנושא מסוים ולא להרבה נושאים. ***שתי השורות הבאות לא נלמדו. לבסוף יש להריץ את הפלוט.

```
# Choose number of topics
library(ldatuning)
```

```
result <- FindTopicsNumber(
  dtm,
  topics = seq(from = 1, to = 10, by = 1),
  metrics = c("Griffiths2004", "CaoJuan2009", "Arun2010", "Deveaud2014"),
  method = "Gibbs",
  #VEM/Gibbs
  control = list(seed = 10, alpha = 0.1),
  mc.cores = 2L,
  verbose = TRUE
)
FindTopicsNumber_plot(result)
```



נרץ את אלגוריתם LDA על TDM, לחלק ל- 5 נושאים מכיוון שהוא נק' המינימום ומקסימום בגרפים, אותם שאר הנתונים כמו בבחירה למעלה. כדי לראות אילו מילים מופיעות בכל נושא, נרץ את הפונק' terms של מודל LDA עם 10 מילים שמאפיינות כל טופיק.

```
##run LDA with the recommended number of topics
```

```
lda <- LDA(dtm, k = 5, control=list(seed=10,alpha=0.1),method = "Gibbs")
#lower alpha, we assume documents contain fewer topics
termsByTopic <- terms(lda, 10)
termsByTopic
```

```
#other options
```

```
#lda_VEM <- LDA(dtm, k = 5, control=list(seed=10)) # find k topics, method = "VEM"
#lda_Gibbs <- LDA(dtm, k = 5, control=list(seed=10),method = "Gibbs") # method = "Gibbs" (alpha = 50/k)
```

ניתן לראות את ההסתברויות אם נפנה לפונק' posterior של LDA ונושאים. כלומר לאחר הרצה נראה שיש 62% שהמסמך הראשון שייך לנושא 1. כך נדע אילו מסמכים מובהקים לאיזה נושא.

נתן לראות את ההסתברויות של כל מילה מבחינת שייכות לנושא מסוים.

```
posterior(lda)$topics[1:10,] #topic distribution for each document (ten first documents)
posterior(lda)$terms[,1:10] # term distribution among topics
head(tweets$text, 4)
```

נרץ את הפונק' topics על מודל LDA וכך נדע לאיזה נושא המסמך השתייך. לאחר מכן נרץ את פונק' table כדי לראות את הסיכום. למשל, בנושא 2 יש 11 מסמכים. נבצע קוסמטיקה לכותרות ע"י שינוי השמות. למשל נושא 1 ייקרא "פלסטין" וככה הלאה.

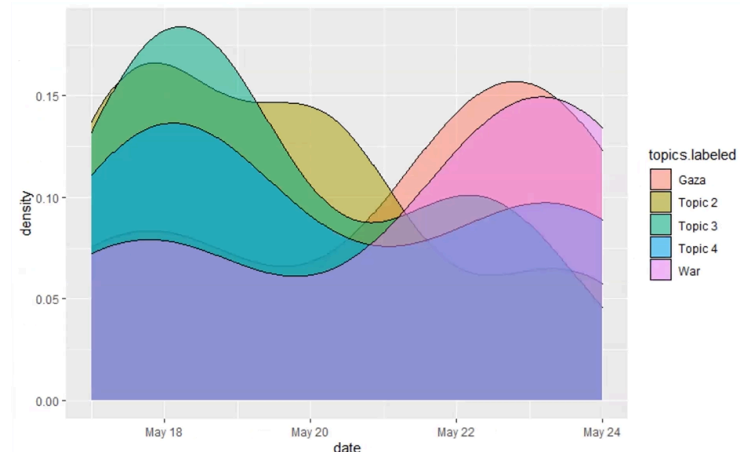
```
#check for lda_VEM, lda_Gibbs
```

```

topics <- topics(lda)
topics #assignment of term to a topic
table(topics) #number of terms in each topic
library(dplyr)
topics.labeled <- recode(topics, '1'='Topic 1','2'='Topic 2','3'='Topic 3', '4'='Topic
4', '5'='Topic 5')
table(topics.labeled)

```

יצירת הגרפים עם התאריך- בדיקה האם נושא מסוים מופיע הרבה בתאריך מסוים. נבנה DF חדש, שיכיל את העמודה תאריך של תאריך הציוץ ועמודת הנושא. נייצר גרף של `geom_density`. כשיש הרבה נושאים, התצוגה לא הכי ברורה.

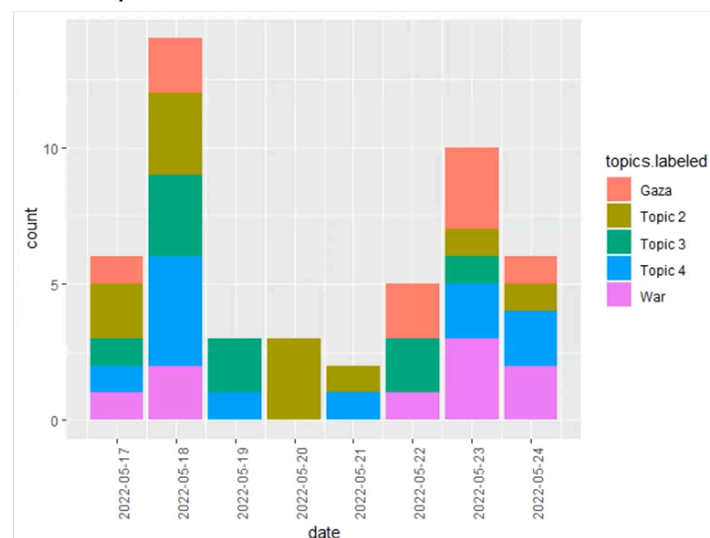


```

topics.by.date <- data.frame(date=as.Date(tweets$created_at),
topic=factor(topics.labeled))
library(ggplot2)
#density plot
ggplot(topics.by.date, aes(date, fill = topics.labeled)) +
  geom_density(alpha = 0.5) # alpha is a transparency param

```

נייצר בר פלוט ונראה כמה פעמים נושא מופיע בתאריך מסוים. למשל, נושא 2 הופיע בתאריך 20.5, אך לא הופיע בתאריך 22.5. שורה ראשונה בנייה של הצירים, שורה שנייה יצירת הגרף, שורה שלישית, פירוש ציר x, שורה רביעית, הטיית הערכים בציר x כדי שיהיו קריאים.



bar plot


```
ggplot(topics.by.date, aes(date, fill = topics.labeled)) +
  geom_bar(position = "stack")+ # position = "stack" - atop one another
  scale_x_date(date_breaks = "days") + #add breaks by day
  theme(axis.text.x = element_text(angle = 90, hjust = 1))
```

קריאת הנתונים מתוך אקסל לאחר הניקיון ואת הנושאים שאליו השתייכו המסמכים.
נשתמש בספריית qdap בגלל שנוח להמיר את הקורפוס ל- DF.

#create a data frame with tweet, date and topic

```
library(qdap)
cleaned.df <- as.data.frame(tweetCorpus)
```

נבנה DF חדש עם עמודת הטקסט המקורית, תאריך הציוץ, הנושא וקבלת הטקסט לאחר הניקיון.
נשמור את הקובץ.

```
tweets.with.topic <- data.frame(text = tweets$text, created =
  as.character(tweets$created_at),
  topic = as.character(topics.by.date$topic),
  cleand_text = cleaned.df$text)
write.csv(tweets.with.topic, 'tweets_with_topic_pop_25052022.csv')
```

שיעור 10- למידה לא מופקחת – 03.06.2022: Topic Modeling

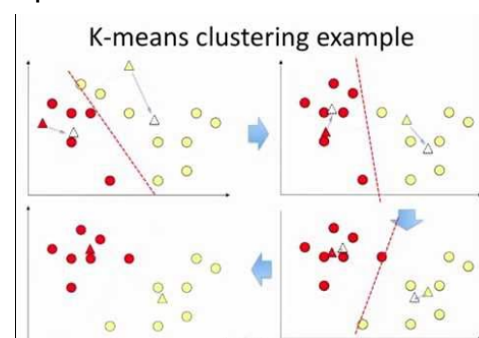
המטרה- מחלקים את המסמכים לקבוצות על בסיס דמיון בין מסמכים. בניית האשכולות נעשית ע"י הוספת מסמכים. כאשר נוספים מסמכים נוספים נעשה חיפוש של מונחים דומים וכך נקבעים הקבוצות. הרעיון- חישוב מרחקים בין מסמכים. אם המרחק קטן, המסמכים דומים.

יש שני אלגוריתמים לביצוע של ניתוח אשכולות:

1. K-Means - אשכול מסמכים ל- K אשכולות שאינם חופפים, כך שהאשכולות יהיו הומוגניים אך אינם דומים לאשכולות אחרים. יש אוסף של מסמכים, כל מסמך הוא נקודה, אנו רוצים לחלק את המסמכים לנקודות. נבחר רנדומלית שני מסמכים, נעבור על כל המסמכים ונבדוק למי הוא יותר דומה. לאחר מכן נעדכן שהוא שייך לאחד האשכולות. לאחר מכן נחשב את הממוצע של כל המסמכים בקבוצה מסוימת ונעדכן את המרכז (המשולש הלבן). נעשה בדיוק את אותו הדבר (עכשיו יש מרכז חדש) עד שמגיעים למצב שאין שיפור במרחק. בשלב זה נעצור.

חסרונות- צריך להחליט על מס' הקבוצות מראש.

כדי לחשב נשתמש בחישוב מרחק אוקלידי בין 2 נקודות או קוסינוס סימילרטי.



$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

The graph shows the relationship between supply and transparency for two documents, Document A and Document B. The x-axis is labeled 'supply' and ranges from 0 to 50. The y-axis is labeled 'transparency' and ranges from 0 to 60. Document A is plotted at (40, 60) and Document B is plotted at (50, 20). Both documents are connected to the origin (0,0) by lines. The graph illustrates that Document A has higher transparency for a lower supply compared to Document B.

תהליך כריית טקסט במחלקת HR לדוג':

- הגדירו את המטרה- האם תוכלו לחפש תכונות משותפות בקרב עובדים שנשארו יותר משנה בארגון.
 - איסוף נתונים- יש 50 קו"ח מעובדים שנשארו יותר משנה בארגון. מכל אחד מקו"ח אתה בוחר את תיאור התעסוקה העדכני ביותר.
 - לארגן- תארגן את הטקסט שלך לתוך DTM עם שקלול TF-IDF.
 - מאפיינים- לבצע אלגוריתם מקבצים של k-means.
 - ניתוח - ניתוח החברות באשכול בקורפוס שלך וזהו מונחים אבות טיפוסיים.
 - סיכום- סכמו מונחים אבות טיפוסיים מובילים עבור כל אשכול.
- לאחר שנריץ את K-means נקבל חלוקה לא מאוזנת. נקבל שרובם הועברו לאשכול אחד (מתוך 3). לכן, נבצע comparison cloud להבין מה המילים שמאפיינות כל אשכול ומה שונה בין האשכולות.
2. Spherical K-Means - נשתמש בקוסינוס סימילרטי כדי להגיע לחלוקה יותר מאוזנת בין האשכולות. לאחר הרצת comparison cloud כדי לראות אילו מילים מאפיינות כל אשכול.
- הביקורת- אלגוריתם לא יציב. נקבל תוצאות שונות (תלוי בנקודת ההתחלה).

#Text mining in R

#part 4.2 code Cluster analysis

```
setwd("C:\\Users\\idopo\\OneDrive\\אילון\\כריית\\שולחן העבודה\\בר")
```

("טקסט\שיעורים\שיעור 10

```
#####
#####
```

```
options(stringsAsFactors = F)
```

```

Sys.setlocale('LC_ALL','C')

```

ספריות מיוחדות לשיעור- skmeans ספרייה עבור- Spherical K-Means.

```
library(skmeans)
```

```
library(tm)
```

```
library(wordcloud)
```

```
library(textstem)
```

```
library(stringr)
```

נפעיל את פונק' הניקיון. כדאי את שורת הסטמינג להריץ פעם אחת עם הערה ופעם אחת בלי.

```
# a function to clean the corpus
```

```
clean.data <-function(text){
```

```
corpus <- VCorpus(VectorSource(text))
```

```
corpus <- tm_map(corpus, content_transformer(tolower))
```

```

corpus <- tm_map(corpus, removeWords, c(stopwords('english'),
"customer","service","customers","calls"))
corpus <- tm_map(corpus, removePunctuation)
corpus <- tm_map(corpus, removeNumbers)
corpus <- tm_map(corpus, stripWhitespace)
corpus <- tm_map(corpus, stemDocument)

return(corpus)
}

```

נקרא את הקובץ כ- csv. לעיתים יש תווים מיוחדים ולכן נשאר רק עם תווים של ASCII (למשל, הורדת אימוג'י). נוסף ניקיון נוסף של מילים שצמודות אחת לשנייה עם פסיק לרווח. לאחר כל הפעולות, נפעיל את פונק' הניקיון ונדפיס את 5 השורות הראשונות.

```

resumeDf<-read.csv('Resumes.csv', header = T)
resumeDf$text <- iconv(resumeDf$text, "latin1", "ASCII", sub=" ")
resumeDf$text <- str_replace_all(resumeDf$text, "[[:punct:]]", " ") #replace
punctuation marks with space
#create the corpus and clean it
resumeCorpus <- clean.data(resumeDf$text)

for(i in 1:5){
  print(resumeCorpus[[i]][1])
}

```

בניית DTM במשקל TF-IDF. כדי שהמטריצה לא תהיה sparse נוכל להוריד את המילים שמופיעים רק פעם 1.

```

#create a DTM
resumeDTM <- DocumentTermMatrix(resumeCorpus,control = list(weighting =
weightTfIdf))
inspect(round(resumeDTM,4)[1:5,1:5])
#resumeDTM <- removeSparseTerms(resumeDTM, .98) # 0.98 - the maximal
allowed sparsity per term.

```

כדי להריץ את פונק' K-Means נקבע את הסיד. נוסף את ה- DTM ונחלק ל- 3 אשכולות. את התוצאה נשים באובייקט resumeClusters. נריץ את האובייקט ביחס לקלאסטר. נראה את התוצאה עבור כל אשכול לאיזה מסמך הוא שייך. כמעט כל המסמכים שייכו לאשכול 3. ניתן יהיה לראות זאת בבר פלוט.

```

#cluster to 3 groups - K-Means
set.seed(1)
resumeClusters<-kmeans(resumeDTM,3)
resumeClusters$cluster
#partialCenters <- resumeClusters$centers[1:3,1:12]
#c1 <- which(resumeClusters$centers[1,]>0) #center 1 terms

# plotting the total number of documents in each cluster
barplot(resumeClusters$size, main='k-means')

```

לכן, נריץ את ספריקל k-means, נקבע את סיד 1. כדי לבנות הבר פלוט, נריץ את הפונק' table על הקלאסטר. נקבל תוצאות דומות באופן יחסי.

```

##Spherical K-Means Clustering
#the distance calculation is based on cosine similarity

#hard clustering
set.seed(1)

```

```
resumeClusters <- skmeans(resumeDTM, 3)
barplot(table(resumeClusters$cluster), main='Spherical k-means')
```

ל- DF שיצרנו נוסיף עמודה שנקראת קלאסטר. בקובץ קו"ח נוצרה העמודה.

```
resumeClusters$cluster
resumeDf$cluster <- resumeClusters$cluster
#centers.s <- resumeClusters$prototypes
```

אלגוריתם k-means הוא קשיח. ז"א שכל מסמך שייך לאשכול אחד בלבד. לעומתו האלגוריתם skmeans הוא לא קשיח. הוא מאפשר לשייך כל מסמך לכמה אשכולות. נקבע אם הפרמטר הוא fuzzy או soft לפי הפרמטר m. אם הוא 1 זה מסמך קשיח, אם נעלה, הוא יאפשר fuzziness.

```
#fuzzy/soft clustering. The m parameter is the fuzziness of the clusters.
#resumeClustersFuzzy <- skmeans(resumeDTM, 3, m = 1.2)
#resumeClustersFuzzy$membership
```

נייצר את הטבלאות וענני המילים. נבצע אגרגציה כדי לאחד את כל הטקסטים עבור כל אשכול. נריץ את פונק' paste ונחבר את קו"ח לפי כל אשכול.

```
## analyze clusters (run for each clustering algorithm separately)
#aggregate the original text by cluster
dataForAnalysis <- aggregate(resumeDf$text~resumeClusters$cluster,
                             FUN = paste,
                             collapse = " ")
```

ננקה את הטקסטים מכיוון כשבנינו את האגרגציה, עשינו את זה על הטקסטים הראשונים ולכן זה לא נקי.

```
#clean the data
dataForAnalysisCorpus <- clean.data(dataForAnalysis[,2])
```

נייצר TDM לפי משקל TF-IDF ונהפוך אותו למטריצה. נסתכל על המטריצה.

*****לא נדע מתי להפעיל DTM או TDM, ברגע שזה לא יצלח נסתכל על העזרה.**

```
#create TDM
dataForAnalysisTdm <- TermDocumentMatrix(dataForAnalysisCorpus,
                                           control=list(weighting=weightTfIdf))
dataForAnalysisTdm.m <- as.matrix(dataForAnalysisTdm)
```

```
#inspect the DTM
dataForAnalysisTdm.m[1:10,1:3]
```

נשנה את הכותרות כדי לתת שם יותר משמעותי ונריץ comparison cloud על המטריצה. ניתן להוסיף שורה של "dev.new" לפי השורה של הענן כדי שהענן ייפתח בחלון חדש.

```
#change column names
colnames(dataForAnalysisTdm.m) <- c("Cluster 1", "Cluster 2", "Cluster 3")
```

```
#comparison cloud
#windows()
comparison.cloud(dataForAnalysisTdm.m,
                 max.words = 50,
                 random.order = FALSE,
```

```
colors = c('red','blue', 'green'), title.size=1)
```

כענת נוכל לייצר טבלה לפי מיון עם מס' המילים הגבוה. נייצר את זה כ- DF המציג עבור כל אוסף מילים, את המילים עם המשקל הכי גבוה בכל אשכול.

```
#top terms
```

```
data.frame (cluster1 =  
names(sort(dataForAnalysisTdm.m[,1],decreasing=T)[1:6]),  
            cluster2 =  
names(sort(dataForAnalysisTdm.m[,2],decreasing=T)[1:6]),  
            cluster3 =  
names(sort(dataForAnalysisTdm.m[,3],decreasing=T)[1:6]))
```

לבסוף, נוסיף את העמודה לקו"ח המקוריים, לשמור כקובץ csv.

```
#write the result to csv
```

```
resumeDf$cluster <- resumeClusters$cluster  
write.csv(resumeDf, "Resume with cluster 2021.csv")
```

שיעור 11- למידה מופקחת (סיווג מסמכים) - Supervised Learning 10.06.22

אלגוריתם של חיזוי. למשל, נרצה לחזות את הרגש של פוסט מסוים. זיהוי האם מסמך מסוים הוא ספאם. ניתן לסווג ספרים / כתבות / סרטים.

בסיווג מסמכים, שדה המטרה (המשתנה התלוי) הוא בינארי או קטגוריאלי. כלומר, מסווגים למשהו שהוא כן או לא, קבוצה 1 או 2. לעומת זאת בעיות רגרסיה, אלו בעיות שנרצה לחזות שמשתנה המטרה הוא רציף. למשל, לחזות את הציון במבחן.

יש לנו דאטה עם עמודה של טקסט ועוד עמודה שהיא ה-class (המשתנה התלוי) עם תשובה של כן או לא (למשל). נבנה DTM, נהפוך את כל מילה לפיצ'ר עד שנקבל תשובה של כן או לא. לאחר מכן נריץ אלגוריתם סיווג שנקרא cv.glmnet עם פונק' של dtm עם ה-class. נקבל מודל עם טקסט חדש ונקבל תוצאה. השוני בסיווג מסמכים הוא שאנחנו בונים DTM על בסיס מילים. אבל בגלל שאנחנו משתמשים באימון וטסט, יהיו מילים שונות ולכן צריך מילים זהות ולכן נוכל להשתמש אך ורק במילים שהיו לנו באימון.

Clickbait Case Study

כאשר בעל אתר מפרסם קישור עם כותרת שמעודדת אנשים ללחוץ כדי לראות יותר, מבלי לספר להם מידע רב על מה שהם יראו. Clickbait משמיט בכוונה מידע חיוני או מגזים בפרטים של סיפור כדי לגרום לו להיראות כמו עניין גדול ממה שהוא באמת. מטרה - זיהוי והגבלת קליקבייט.

למשל, בפייסבוק, יש פוסטים של פרסומות ולפעמים נצטרך ללחוץ עליהם. המטרה של פייסבוק למגר את התופעה. הם בנו אלגוריתם סיווג שמאתר Clickbait.

במסד הנתונים של השיעור יש את עמודת המטרה y שבו נדע אם יש Clickbait או לא- 0 או 1.

תהליך כריית הטקסט:

- הגדרת המטרה- יצירת אלגוריתם סיווג מסמכים המזהה קליקבייט. הערכת האלגוריתם והצעה לשיפור הביצועים שלו.
- איסוף נתונים- המודל יבנה באמצעות 3000 כותרות. המשתנה "y" הוא 1 אם הכותרת הגיעה ממקור clickbait ו- 0 אם המקור נחשב לחדשות.
- ארגון- הטקסט יופרד לקבוצות אימון ומבחן. המודל יבנה באמצעות ההכשרה ויוערך על סמך מערך המבחנים.
- מאפיינים- בניית DTM משוקלל TF-IDF מנתוני האימון. ה-DTM המקורי ישמש לבניית ה-DTM במבחן.
- מטריצות ניתוח DTM הן דלילות ביותר. כתוצאה מכך, אימון רגרסיית לאסו של GLMNet. נעריך את הדיוק של אלגוריתם הסיווג ונקבע את מונחי ה-clickbait המשפיעים ביותר.
- לסיכום- הכרת המונחים המובילים המאותתים קליקבייט. נוכל להמליץ על דרכים לשיפור הדיוק של האלגוריתם.
- יש מס' מדדי הערכה, בשיעור נראה AUC לפי קונפיזון מטריקס. ניקח את כל התוצאות שצדקנו ונחלק ביניהן.
- לאחר שבנינו את המודל, נבנה את הטסט לפי הנתונים של האימון. נבדוק את הקונפיזון מטריקס ונגלה שהוא פחות טוב משל האימון, כלומר, יש יותר טעויות. בהמשך, נבדוק אחרי כמה מילים יש בכותרת קליקבייט.

שיטות אפשריות לשיפור המודל:

- הנדסת תכונות- יצירת קלטות נוספות עבור מס' המילים בכל כותרת. למשל, הוספת חותמת זמן או מקור.
- טוקניזציה- ניסוי עם טוקניזציה של בי-גרם ותלת-גרם כקלט מודל. למשל לא טוב.
- עונש אלפא - התאם את פרמטר העונש כדי לאזן את מספר הכניסות והדיוק. יותר כניסות <- זמן חיזוי ארוך יותר.
- ערימת חיזוי ומידול אנסמבל - שילוב מודלים נוספים בשיטות שונות. כגון lasso and naïve Bayes למודל אחרון מורכב יותר.
- יותר נתונים- הגדל את מספר הכותרות והמקורות.

#part 5.1 code Document Classification

```
setwd("C:\\Users\\idopo\\OneDrive\\אילן\\כריית\\שולחן העבודה\\בר אילן\\כריית\\  
11(טקסט\\שיעורים\\שיעור 11
```

```
library(tm) #preprocessing, term document matrix, corpus
library(Matrix) #provides memory efficient methods for manipulating sparse
matrices.
```

```
library(glmnet) #regression model
library(caret) #data partitioning, confusion matrix
library(pROC) #ROC curves
library(ggthemes) #visualization
library(ggplot2) #visualization
```

```
options(stringsAsFactors = F)
```

```
#cleaning function
```

```
headline.clean <-function(text){
  corpus <- VCorpus(VectorSource(text))
  corpus <- tm_map(corpus, content_transformer(tolower))
  corpus <- tm_map(corpus, removeWords, stopwords('english'))
  corpus <- tm_map(corpus, removePunctuation)
  corpus <- tm_map(corpus, removeNumbers)
  corpus <- tm_map(corpus, stripWhitespace)
```

```
return(corpus)
```

}

```
#Partition the data
```

#The createDataPartition function selects random number of rows based on the dependent variable distribution.

```
headlines<-read.csv('all 3k headlines.csv',encoding="UTF-8")
```

```
headlines$headline <- iconv(headlines$headline, "UTF-8", "ASCII", sub="")
```

```
#convert UTF-8 to ASCII
```

נחלק לאימון וטסט. בניגוד לקורס ב-R שהשתמשנו בפונק' שבוחרת אינדקסים רנדומלית. נשתמש בפונק' שנקראת createDataPartition הדואגת לשמור אינדקסים מאוזנים.

```
set.seed(1)
```

```
train<-createDataPartition(headlines$y,p=0.6,list = F) # p -> the percentage of
data that goes to training. #list = F -> the result will be in a matrix and not a list
```

```
train.headlines<-headlines[train,]
```

```
test.headlines<-headlines[-train,]
```

כדי לבנות מודל צריך לנקות.

#clean the training set

```
clean.train<-headline.clean(train.headlines$headline) #clean the training set
```

```
for(i in 1:5){
  print(clean.train[[i]][1])
}
```

בונים DTM לפי משקל TF-IDF.

```
#create a DTM
```

```
train.dtm <- DocumentTermMatrix(clean.train,control = list(weighting =  
weightTfIdf)) #the orig DTM is null in this case  
train.dtm  
inspect(train.dtm)
```

נהפוך את האימון למטריצה.

```
train.matrix<-as.matrix(train.dtm)
```

```
#change the object into a sparse matrix (memory efficient)
```

```
train.matrix<-Matrix(train.matrix, sparse = T) #sparse = T will automatically  
become true if more than half the values are 0  
dim(train.matrix)
```

בניית המודל ע"י פונק' `cv.glmnet` על מטריצת האימון ואת שדה המטרה `y`.

- `family='binomial'` - הבהרה למודל שאנחנו בבעיית סיווג.

- `nfolds = 10` חלוקה ל 10 CV.

```
#build the model
```

```
Set.seed(1)
```

```
cv<-cv.glmnet(train.matrix, y=as.factor(train.headlines$y),  
alpha=0.8,  
family='binomial',  
nfolds = 10,  
intercept = F,  
type.measure = 'class')
```

נשתמש במודל על מנת לחזות את התוצאות. ב `type` ניתן לרשום `class` או `response`.
ההבדל האם אנחנו רוצים שהחיזוי יחזיר הסתברות (`response`) או תוצאת 0 או 1 (`class`).

```
#apply the model to the training set
```

```
preds<-predict(cv,train.matrix,type='response',s=cv$lambda.1se)  
head(preds,10) #1 = clickbait
```

הערכת המודל ע"י בניית `roc` עם פונק' שתגיד מה התוצאה האמיתית לעומת תוצאת החיזוי.

```
#Evaluate the model
```

```
#create a roc curve
```

```
train.auc<-roc(train.headlines$y,as.numeric(preds))  
train.auc  
plot(train.auc)
```

יצירת קונפיזון מטריקס. ניתן את תוצאות חיזוי של הפרדיקציה ואת הוצאות האמיתיות.

```
#create the confusion matrix
```

```
confusion.train <-  
confusionMatrix(as.factor(ifelse(preds>=0.5,1,0)),as.factor(train.headlines$y))  
confusion.train
```

ביצוע אותן פעולות על הטסט עם נתונים שהוא לא ראה. לכן קודם כל צריך לנקות את הטקסט.

```
#Test the model on new data
```

```
#clean the test set
```

```
clean.test<-headline.clean(test.headlines$headline)
```

נגדיר במילון אך ורק את המילים שיש באימון. נהפוך אותו למטריצה.

```
#create the DTM
```

```
#match the matrix to contain same words as the training matrix
```



```
test.dtm <- DocumentTermMatrix(clean.test, control =
list(dictionary=Terms(train.dtm), weighting = weightTfIdf))
test.dtm
test.matrix<-as.matrix(test.dtm)
test.matrix<-Matrix(test.matrix)
```

נריץ את החיזוי.

```
#predict on the test data
preds<-predict(cv,test.matrix,type='response',s=cv$lambda.min)
head(preds.test)
```

```
#create a data frame that contains the doc row and the prediction
#headline.preds<-data.frame(doc_row =
rownames(test.headlines),class=preds[,1])
#head(headline.preds)
```

נחשב את ה-AUC.

```
#calculate AUC
test.auc<-roc(test.headlines$y,as.numeric(preds))
test.auc
plot(test.auc)
```

השוואה של הגרפים.

```
#compare train and test
plot(train.auc,col="blue",main="RED = test, BLUE = train", adj=0) #adj=0, title
aligned left
plot(test.auc, add=TRUE, col="red", lty=2)#lty=2, line is dashed
```

ניצור קונפיזון מטריקס גם לנתוני הטסט.

```
#confusion matrix
confusion.test <-
confusionMatrix(as.factor(headline.preds[,2]),as.factor(test.headlines$y))
confusion.test
```

לאחר בניית מודל החיזוי, נרצה לבדוק אלו מילים הכי השפיעו על המודל ע"י פונק' מקדם-
coefficients

```
#finding the most impactful words
#model coefficients
glmnet.coef<-as.matrix(coef(cv, s='lambda.min'))
```

נהפוך את זה ל- DF מטעמי נוחות. העמודה הראשונה היא מילה והמילה השנייה היא
מקדם.

```
#add column names (words,glmnet_coefficients)
glmnet.coef<-data.frame(words=row.names(glmnet.coef),glmnet_coefficients=
glmnet.coef[,1])
```

נמיין לפי עמודת מקדם בסדר יורד.

```
#sort in decreasing order
glmnet.coef<-glmnet.coef[order(glmnet.coef$glmnet_coefficients, decreasing
= T),]
```

על מנת להציג את הנתונים בגרף נהפוך את המילים לפקטור. נוסיף את
בשביל שמירת הסדר.

```
#convert words to factor (for ggplot)
```

```
glmnet.coef$words<-factor(glmnet.coef$words,levels=unique(glmnet.coef$words))
```

צפייה בסיכום הנתונים לפי ממוצע, חציון וכו'.

#1st and 3rd quartiles are 0 because the lasso regression forced many of the term coefficients to 0.

```
summary(glmnet.coef$glmnet_coefficients)
```

נבדוק כמה מקדמים הם חיוביים / שליליים / ניטרליים.

#number of positive coefficients

```
length(subset(glmnet.coef$glmnet_coefficients,glmnet.coef$glmnet_coefficients>0))
```

#number of negative coefficients

```
length(subset(glmnet.coef$glmnet_coefficients,glmnet.coef$glmnet_coefficients<0))
```

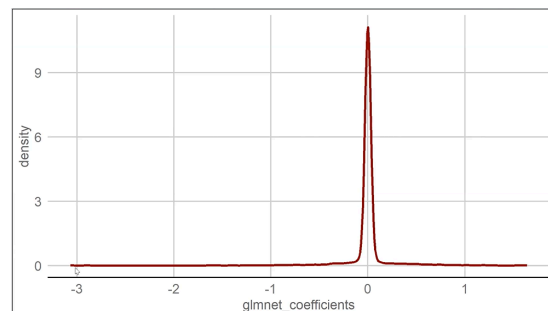
#number of 0 coefficients

```
length(subset(glmnet.coef$glmnet_coefficients,glmnet.coef$glmnet_coefficients==0))
```

ניצור את הגרף עם המקדם.

#create the density plot for word coefficients

```
ggplot(glmnet.coef,aes(x=glmnet_coefficients)) +  
  geom_line(stat = 'density', color='darkred',size=1) +  
  theme_gdocs()
```



צפייה ב- 10 המונחים שהכי השפיעו. נשתמש בפונק' rbind לצפייה באלו שהכי נצפו ובפונק' tail לאלו שהשפיעו הכי פחות. ניצור עמודה חדשה שנקראת impact לבדיקת חיוביות / שליליות.

#inspect the meaningful terms

```
top.coef<-rbind(head(glmnet.coef,10),tail(glmnet.coef,10))
```

```
top.coef$impact<-ifelse(top.coef$glmnet_coefficients>0,"Positive","Negative")
```

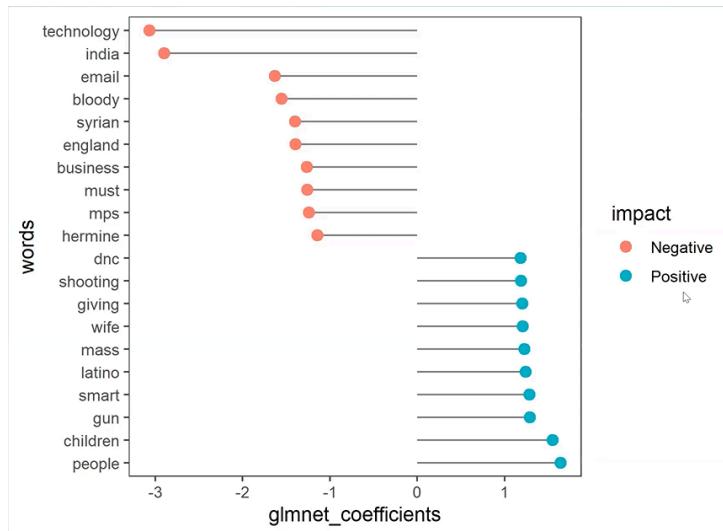
#positive=clickbait

```
top.coef
```

נריץ את ggplot, נוסיף את פונק' geom_segment המוסיפה קווים. נוסיף את geom_point בשביל הנקודות. פונק' theme_few תראה את הגרף בצורה יותר פשוטה.

```
ggplot(top.coef, aes(x=glmnet_coefficients, y=words)) +  
  geom_segment(aes(yend=words), xend=0, colour="grey50")+  
  geom_point(size=3, aes(colour=impact)) + theme_few()
```

#geom_segment draws a straight line between points (x, y) and (xend, yend)



שיעור 12- סיווג מסמכים- השוואת שיטות- 17.06.22:

שילוב בין פיצ'רים טקסטואליים לבין פיצ'רים מספריים.
 אינטואיציה- מציאת מילים קשורות על סמך התרחשות משותפת של מילים. לאחר מכן ניתן לחשב דמיון על מסמכים בעלי משמעות דומה ולא דומה ע"י דמיון טקסט בפועל. היכולת להבין מסמכים חדשים גם אם יש טרמינולוגיה חדשה.
 במקום לבנות DTM נמדל כל מילה כייצוג ווקטורי. התוצר של כל מילה אמור להיות ווקטור. למשל למילים איש ואישה יהיה ווקטור מאוד דומה.
מטריצת מופע משותף מבוסס חלונות:

- הופעה משותפת- ההתרחשות המשותפת של צמד מילים w_1 ו- w_2 היא מס' הפעמים שהן מופיעות יחד בחלון הקשר.
- חלון ההקשר- מצוין ע"י מס' והכיוון.
- Word2Vec היא גישה שעוזרת לנו להשיג וקטורים דומים עבור מילים דומות. מילים הקשורות זו לזו ממפות לנקודות הקרובות יותר זו לזו במרחב ממדי גבוה.
- אשפוז חוזר של מטופלים:

- הגדרת המטרה- האם מטופל יחזור לבית החולים תוך 30 יום?
- איסוף נתונים- נאספו מדגם של 8500 רשומות של חולי סוכרת, כולל הערות שחרור.
- ארגון- חלוקת הנתונים לקבוצות אימון וטסט.
- תכונות- גישות Bag Of Words & Word2Vec; ייעשה שימוש בתכונות נוספות.
- ניתוח- עם גרסיה של לאסו ו- SVM.
- סיכום- שימוש במודל הטוב ביותר על רישום מטופל חדש, המלצה אם המטופל מועמד לאשפוז חוזר.

הערה- רוני לא הסבירה על 12.1

```
library(text2vec)
library(occurrence)
library(qdap) #convert corpus to data.frame
library(ggplot2)
vignette(text2vec)
## read data
ermany <- read.csv("Resumes.csv")
ermany <- read.csv("test.csv") #small example
```

a function to clean the corpus

```
clean.data <-function(text){
  corpus <- Vcorpus(VectorSource(text))
```

```

corpus <- tm_map(corpus, content_transformer(tolower))
corpus <- tm_map(corpus, removeWords, c(stopwords('english'),
"customer", "service", "customers", "calls"))
corpus <- tm_map(corpus, removePunctuation)
corpus <- tm_map(corpus, removeNumbers)
corpus <- tm_map(corpus, stripWhitespace)
corpus <- tm_map(corpus, stemDocument)
return(corpus)
}
resumeCorpus <- clean.data(ermanya$text)

```

יצירת Word2Vec.

```

# create word tokens and vocabulary
processedDocuments <- as.data.frame(resumeCorpus)
it <- itoken(processedDocuments$text, progressbar = FALSE)
#v <- create_vocabulary(it) #run for small example
v <- create_vocabulary(it) %>%
  prune_vocabulary(doc_proportion_max = 0.1, #maximum proportion of
documents which should contain term
  term_count_min = 5)
vectorizer <- vocab_vectorizer(v)

```

```

# Term-co-occurrence matrix
set.seed(1)
tcm <- create_tcm(it, vectorizer, skip_grams_window = 5L)
tcm[1:10,1:4]
#as.matrix(tcm) #run only for small example

```

```

# fit text2vec model
glove <- GlobalVectors$new(rank = 2, x_max = 10) #length 2 vectors
#fit Glove model to input matrix
word_vectors_main <- glove$fit_transform(tcm, n_iter = 20)
word_vectors_context <- glove$components
word_vectors <- word_vectors_main + t(word_vectors_context)
plot.data <- data.frame(word_vectors)
ggplot(plot.data,
  aes(x = X1, y = X2)) +
  geom_text(
    label=rownames(plot.data),
    nudge_x = 0.01, nudge_y = 0.01
  )

```

```

## Wikipedia example
## from: https://cran.r-project.org/web/packages/text2vec/vignettes/glove.html
library(data.table)
library(lsa) # for cosine
wiki <- readLines("ermanya.dat", n = 1, warn = FALSE)

tokens <- space_tokenizer(wiki) #faster but less general
it <- itoken(tokens, progressbar = FALSE)
vocab <- create_vocabulary(it)

```



```
## partition
set.seed(1)
trainIDs<-createDataPartition(diabetesDF$readmitted,p=0.6,list = F) # p -> the
percentage of data that goes to training. #list = F -> the result will be in a
matrix and not a list
#createDataPartition attempts to balance the class distributions within the
splits.
train.diabetes<-diabetesDF[trainIDs,]
test.diabetes<-diabetesDF[-trainIDs,]

# another approach
#set.seed(1)
#trainIDs <- sample(1:(dim(diabetesDF)[1]), dim(diabetesDF)[1]*0.6)
#train.diabetes <- diabetesDF[trainIDs,]
#test.diabetes <- diabetesDF[-trainIDs,]
```

הרצת פונק' preprocess על מנת לנקות את הטקסט והכנסתו לקורפוס.

```
#preprocess the training set
diabetesCorpus.train <- preprocess(train.diabetes$diag.text)
for(i in 1:5){
  print(diabetesCorpus.train[[i]][1])
}
```

בניית DTM האימון.

```
#create a train DTM
train.dtm <- DocumentTermMatrix(diabetesCorpus.train)
#1.todo: try with tf-idf, what do you get?
#2.todo: try with unigrams + bi-grams, what do you get?

#train.dtm <- removeSparseTerms(train.dtm, .999)
inspect(train.dtm)
```

הרצה על מנת לנקות + בניית DTM של הטסט.

```
#preprocess the test set
diabetesCorpus.test <- preprocess(test.diabetes$diag.text)
```

בניית DTM על נתוני הטסט על סמך המילים המופיעות באימון.

```
#create a test DTM
test.dtm <- DocumentTermMatrix(diabetesCorpus.test, control =
list(dictionary=Terms(train.dtm)))
inspect(test.dtm)
```

אריזת קטע הקוד הבא כפונק'. נשתמש בפונק' cv.glmnet.
נחזה את המודל על נתונים חדשים- נתוני הטסט.
נבנה קונפיזן מטריקס על מנת לקבל את אחוז הדיוק.

```
# functions -----
lassoPredict <- function(train.mat, test.mat){

  cv <- cv.glmnet(train.mat,
    y = as.factor(train.diabetes$readmitted),
```

```

        alpha=1,
        family='binomial',
        nfolds=10,
        intercept=F,
        type.measure = 'class')

preds <- predict(cv, test.mat,
                 type = 'class', s=cv$lambda.1se)

cm <- confusionMatrix(as.factor(preds),
                      as.factor(test.diabetes$readmitted))

return(cm$overall[1])
}

```

יצירת פונק' נוספת שנקראת svmPredict המקבלת אימון וטסט.
 נבנה מודל שנקרא sv על נתוני הטסט על מנת לקבל חיזוי, נבנה קונפיזון מטריקס על מנת להשוות את נתוני החיזוי לעומת הנתונים בפועל.

```

svmPredict <- function(train.mat, test.mat){
  sv <- svm(train.mat, y= as.factor(train.diabetes$readmitted), gamma = 0.1)

  preds <- predict(sv, test.mat) #na.action = na.exclude

  cm <- confusionMatrix(as.factor(preds),
                        as.factor(test.diabetes$readmitted))

  return(cm$overall[1]) #accuracy
}

```

כעת נריץ את המודל בשיטת BOW לפי לאסו, נהפוך את ה- DTM של האימון והטסט למטריצה. לאחר מכן נריץ את הפונק' lassoPredict שתחזיר את מדד הדיוק.

```

# BoW -----
# lasso regression: y ~ BoW

train.matrix<-Matrix(as.matrix(train.dtm), sparse = T)
test.matrix<-Matrix(as.matrix(test.dtm), sparse = T)
#3.todo - try not to convert to a Matrix, what is the difference?
set.seed(1)
lassoPredict(train.matrix, test.matrix)

```

כעת נבצע את אותה פעולה רק לפי svm.

```

# svm: y ~ BoW
svmPredict(train.matrix, test.matrix)

# naiveBayes: y ~ BoW
# train.mat.scaled <- scale(as.matrix(train.dtm))
# test.mat.scaled <- scale(as.matrix(test.dtm))

```

```
# naiveBayesPredict(train.mat.scaled, test.mat.scaled)
```

נריץ את הפונק' רק על העמודות המספריות ונקבל מדד דיוק גבוה יותר 0.73 לעומת 0.67.
לאחר מכן נבצע אותה פעולה על svm ונקבל 0.67 לעומת 0.66

```
# other variables -----
```

```
# lasso regression: y ~ numeric variables
```

```
lassoPredict(as.matrix(train.diabetes[,1:132]), as.matrix(test.diabetes[,1:132]))
```

```
# svm: y ~ other variables
```

```
svmPredict(as.matrix(train.diabetes[,1:132]),  
           as.matrix(test.diabetes[,1:132]))
```


כעת, נאחד בין BOW לבין ערכים אחרים. נשתמש בפונק' cbind המאחדת את העמודות. נבצע זאת גם לאימון וגם לטסט. קיבלנו תוצאה של 0.74. כלומר, המודל השתפר כאשר משלבים בין פיצ'רים טקסטואליים לפיצ'רים מספריים. נבצע את אותה פעולה בדיוק ל- svm ונקבל 0.65, כלומר תוצאות פחות טובות.

```
# combine attributes -----
cbind(c(1,2),c(3,4)) #cbind example
set.seed(1)
lassoPredict(cbind(train.matrix, as.matrix(train.diabetes[,1:132])),
              cbind(test.matrix, as.matrix(test.diabetes[,1:132])))

# svm: y ~ combined variables
svmPredict(cbind(train.matrix, as.matrix(train.diabetes[,1:132])),
            cbind(test.matrix, as.matrix(test.diabetes[,1:132])))

####advanced
##text2vec
train.diabetes$diag.text <- as.data.frame(diabetesCorpus.train)[,2] # change
original text to the cleaned text (for text2vec)
test.diabetes$diag.text <- as.data.frame(diabetesCorpus.test)[,2] #change
original text to the cleaned text (for text2vec)

it <- itoken(train.diabetes$diag.text) # Creates iterators over input objects in
order to create vocabularies
v <- create_vocabulary(it) # This function collects unique terms and
corresponding statistics
#4.todo - add bigrams to the vocabulary, how did it affect the results?
vectorizer <- vocab_vectorizer(v) # Defines on how to transform list of tokens
into vector space

tcm <- create_tcm(it, vectorizer, skip_grams_window = 5L)
glove <- GlobalVectors$new(rank = 100, x_max = 10) #x_max = maximum
number of co-occurrences to use in the weighting function
word_context <- glove$fit_transform(tcm, n_iter = 100)
word_vectors <- glove$components + t(word_context)
word_vectors[1:100,1:10]
dim(word_vectors)

train.dtm <- create_dtm(it, vectorizer)
dim(train.dtm)
train.dtm <- as.matrix((train.dtm/Matrix::rowSums(train.dtm)) %*%
t(word_vectors))
# doc2vec - calculate an average vector of all the words

test.it <- itoken(test.diabetes$diag.text)
test.dtm <- create_dtm(test.it, vectorizer) # vectorizer contains train
vocabulary
test.dtm <- as.matrix((test.dtm/Matrix::rowSums(test.dtm)) %*%
t(word_vectors))
dim(test.dtm)
# lasso regression: y ~ text2vec
```

```
set.seed(1)
lassoPredict(train.dtm,test.dtm)
```

```
# svm: y ~ text2vec
svmPredict(train.dtm, test.dtm)
```

#5.todo - try text2vec + other variables, what is the accuracy? which model gave the best result?

##matrix example

```
DTM = matrix(
  c(1, 1, 1, 0, 1, 1, 0, 1), # the data elements
  nrow=2,                    # number of rows
  ncol=4,                    # number of columns
  byrow = TRUE)             # fill matrix by rows
```

```
WVEC = matrix(
  c(0.25, 0.5, 0, 0, 0, 0.25, 0, 0.15, 0.15, 0, 0.2, 0), # the data elements
  nrow=3,                    # number of rows (word2vec length)
  ncol=4,                    # number of terms
  byrow = TRUE)             # fill matrix by rows
```

```
newMatrix <- as.matrix((DTM/rowSums(DTM)) %*% t(WVEC)) #*%*% dot
product
```